

BigClaw AI — Design Basis and Description

Autonomous Investment Research & Portfolio Management System

Field	Detail
System Name	BigClaw AI
Document No.	DBD-AI-001 Rev. 1
Version	July 22, 2026 (four-sleeve cutover + Commando radar/doctrine; shadow OpenRouter/Grok Judge still experimental — not production)
Author	Curtis Biggs
Hostname	BigClaw
Dashboard	https://bigclaw.grandpapa.net
GitHub Review Copy	https://github.com/cbiggs1-tech/bigclaw-codebase

Table of Contents

- [1. System Description](#)
- [2. Hardware Platform](#) - 2.1 Hardware Specifications - 2.2 Development Access
- [3. Software Architecture](#) - 3.1 System Diagram - 3.2 Codebase - 3.3 Services
- [4. Database Schema](#) - 4.1 Tables
- [5. Portfolio Management Philosophy](#) - 5.1 Portfolio Definitions - 5.2 Portfolio Parameters - 5.3 LLM Discretionary / Commando - 5.4 Four-Sleeve Live Book (July 2026)
- [6. Decision Engine & Scoring System](#) - 6.1 Signal Dimensions (20) - 6.2 Style-Specific Signal Weights - 6.3 Score-to-Label Mapping - 6.4 Special Rules - 6.5 Style Gate Checks - 6.6 AI Gate Reasoning - 6.7 Candidate Discovery Screener
- [7. Autonomous Trading Logic & Execution](#) - 7.1 Execution Flow - 7.2 Position Sizing - 7.3 Safety Rules
- [8. Risk Management — Trailing Stops](#) - 8.1 Trail Percentages by Portfolio - 8.2 Stop Lifecycle - 8.3 Intraday Stop Monitor
- [9. Scheduled Operations & Automation](#) - 9.1 OpenClaw Cron Jobs (Weekday) - 9.2 OpenClaw Cron Jobs (Weekly) - 9.3 OpenClaw Cron Jobs (Daily) - 9.4 System Crontab Jobs - 9.5 Disabled Jobs
- [10. Data Integrity & Anti-Hallucination Pipeline](#)
- [11. Data Feeds & External APIs](#)
 - 11.1 Market Data & Pricing
 - 11.2 Options Flow & Institutional Intelligence
 - 11.3 Sentiment & Social
 - 11.4 News & Research
 - 11.5 ARK Invest Tracking
 - 11.6 Economic Calendar & Macro
 - 11.7 Weather & Environment
 - 11.8 Website Frontend
 - 11.9 LLM Providers
 - 11.10 Communication
 - 11.11 Technical Analysis Libraries

- 12. [User Interfaces](#)
 - 12.1 Slack — Primary Interactive Channel
 - 12.2 GitHub Pages Dashboard
 - 12.3 Dashboard Data Files
 - 13. [Logging, Monitoring & Security](#)
 - 13.1 Application Logs
 - 13.2 Trade Logs
 - 13.3 Intraday Stop Check Logs
 - 13.4 API Retry
 - 13.5 Configuration Files
 - 13.6 Security
 - 14. [Key Design Principles](#)
-

1. System Description

BigClaw AI is a fully autonomous investment research agent and paper-trading portfolio manager that runs 24/7 on a single Raspberry Pi 4 Model B. Its purpose is simple and unchanging: gather, synthesize, and analyze market data from every available source — price action, technicals, fundamentals, sentiment, insider activity, and macro signals — then act on that intelligence to manage seven distinct style-specific paper portfolios.

System Functions:

- **Research** — continuously monitors the market without human intervention.
- **Manage** — executes simulated trades inside strict, philosophy-driven rules using Alpaca's paper-trading API (each portfolio begins with \$100,000 virtual cash).
- **Protect** — enforces position limits, trailing stops, concentration caps, and a hard "no-trade" rule outside the 10:00 AM–4:00 PM ET weekday window.

BigClaw reports every morning (market overview), every afternoon (portfolio performance), and in real time (trade alerts) via Slack and the public dashboard at bigclaw.grandpapa.net. It improves itself through weekly research sessions, style-compliance audits, and decision-engine refinements.

Important reminder for anyone who inherits or touches this system: BigClaw is strictly paper trading. It has never and will never execute real-money trades. Any future attempt to connect it to a live brokerage account must be treated as a complete redesign.

2. Hardware Platform

BigClaw runs on a Raspberry Pi 4 Model B. This choice is deliberate: it is low-power, silent, always-on, and inexpensive to replace. The entire system — code, database, logs, and GitHub Pages refresh — lives on this one device. The local Windows copy and GitHub repository exist only as review snapshots; the Pi is the single source of truth.

Development happens primarily through VS Code Remote SSH (key-based ed25519 authentication). The OpenClaw agent runtime and Claude Code CLI are also installed directly on the Pi.

2.1 Hardware Specifications

Component	Specification
Board	Raspberry Pi 4 Model B Rev 1.4
Processor	Broadcom BCM2711, ARM Cortex-A72 (4 cores @ 1.8 GHz)
Architecture	aarch64 (64-bit ARM)
Memory	8 GB LPDDR4
Storage	256 GB USB SSD (228 GB usable, 8% utilized)
Operating System	Debian GNU/Linux 13 (Trixie)
Kernel	Linux 6.12.62+rpt-rpi-v8 SMP PREEMPT
Hostname	BigClaw
Network	Local LAN at 192.168.1.171, SSH alias <code>bigclaw</code>
Power	Continuous operation, auto-restart on failure

2.2 Development Access

Method	Detail
VS Code Remote SSH	Primary development interface from Windows desktop
Claude Code CLI	Installed on Pi via OpenClaw (<code>~/ .openclaw/</code>)
SSH from Windows	<code>ssh bigclaw</code> (key-based auth, ed25519)

3. Software Architecture

The system is deliberately split into two cooperating codebases so that data collection, trading logic, and interactive conversation stay fast and cheap while heavy reasoning is offloaded to cloud LLMs only when needed.

3.1 System Diagram

```
Raspberry Pi 4B - "BigClaw"
├─ systemd: bigclaw.service (always running)
│   └─ bot.py → Slack agent + Claude conversation loop
│       ├── agent.py      - Claude tool-use agent
│       ├── tools/ (15 modules) - market, charts, orders, social, etc.
│       ├── services/     - price oracle, output guardrail
│       ├── portfolio.py  - SQLite ORM for all portfolio data
│       ├── llm_router.py - cost-optimized model routing
│       └─ memory.py      - conversation memory
├─ systemd: openclaw-gateway.service
│   └─ OpenClaw agent runtime (WhatsApp/Slack gateway + cron engine)
│       ├── cron/jobs.json - 14 scheduled jobs
│       ├── workspace/scripts/ - 30+ Python scripts
│       ├── workspace/skills/ - 27 modular agent skills
│       ├── workspace/config/ - portfolio universes, expert overrides
│       └─ workspace/SOUL.md - agent personality & analytical mandate
├─ System crontab (1 job)
│   └─ stop_check.py      - 15-min trailing stop monitor
```

```

├── docs/ → GitHub Pages (bigclaw.grandpapa.net)
│   ├── 10 HTML pages
│   └── data/ - 11 JSON files + chart data (auto-refreshed)
└── src/portfolios.db - SQLite database (source of truth)

```

3.2 Codebase

3.2.1 Core Application — `~/bigclaw-ai/`

The core application is the always-on Slack bot, the portfolio database, and the public dashboard. It runs as `bigclaw.service` under systemd and is the user-facing side of BigClaw.

Slack Bot (`src/bot.py`) — Built on `slack-bolt` with Socket Mode (no inbound webhooks, no public ports). The bot listens for direct messages and channel mentions, routes each message through the Claude agent with full conversation memory, and returns the response. Image-based responses (charts) are uploaded as Slack files. The bot requires three tokens: `SLACK_BOT_TOKEN`, `SLACK_APP_TOKEN`, and `SLACK_SIGNING_SECRET`.

Claude Agent (`src/agent.py`) — Implements a tool-use conversation loop against the Anthropic API. When a user sends a message, the agent passes it to Claude along with the full set of available tools. If Claude invokes a tool, the agent executes it locally, feeds the result back, and continues the loop until Claude produces a final text response. Conversation history is maintained per-channel via `memory.py` so context carries across messages.

Interactive Tools (`src/tools/`) — 36 tools organized across 10 modules that Claude can invoke during conversation:

Module	Tools	What They Do
<code>market.py</code>	<code>get_stock_quote</code> , <code>get_stock_details</code>	Live price, fundamentals, P/E, market cap, sector
<code>charts.py</code>	<code>generate_stock_chart</code> , <code>compare_stocks_chart</code> , <code>generate_macd_chart</code> , <code>generate_rsi_chart</code> , <code>generate_bollinger_chart</code> , <code>generate_monte_carlo_chart</code> , <code>generate_moving_averages_chart</code>	Technical analysis chart generation (uploaded as images)
<code>news.py</code>	<code>get_yahoo_news</code> , <code>search_financial_news</code>	Per-ticker and broad financial news
<code>social.py</code>	<code>get_x_sentiment</code> , <code>get_stocktwits_sentiment</code> , <code>search_reddit_stocks</code> , <code>get_wsb_trending</code>	Social sentiment from X/Twitter, StockTwits, Reddit
<code>predictions.py</code>	<code>search_polymarket</code> , <code>get_polymarket_trending</code>	Prediction market odds
<code>portfolio.py</code>	<code>create_portfolio</code> , <code>delete_portfolio</code> , <code>list_portfolios</code> , <code>view_portfolio</code> , <code>buy_stock</code> , <code>sell_stock</code> , <code>get_transactions</code> , <code>compare_portfolios</code>	Full portfolio CRUD and trade execution
<code>orders.py</code>	<code>set_stop_loss</code> , <code>set_limit_buy</code> , <code>set_limit_sell</code> , <code>view_pending_orders</code> , <code>cancel_order</code>	Pending order management
<code>strategy_analyzer.py</code>	<code>run_analysis_now</code>	Trigger immediate decision engine analysis
<code>technical.py</code>	Additional technical analysis tools	Extended TA capabilities
<code>base.py</code>	<code>set_report_channel</code> , <code>set_autonomous_trading</code>	System configuration

LLM Router (`src/llm_router.py`) — A lightweight cost-optimization layer that routes LLM calls to the cheapest model capable of the task. Routine data summarization goes to Gemini Flash Lite via OpenRouter (~\$0.01/call). Analytical work requiring deeper reasoning is routed to Claude Sonnet (~\$0.20/call). Interactive Slack conversations use Claude Sonnet directly via the Anthropic API. The router exposes `call_openrouter()` for general calls and `summarize_with_flash()` as a convenience wrapper for cheap summarization tasks.

Services (`src/services/`): - **Price Oracle** (`price_oracle.py`) — Single source of truth for all prices system-wide. In-memory cache with 10-minute TTL. Refuses to serve stale data. - **Output Guardrail** (`output_guardrail.py`) — Scans every outbound message for `$$$` price patterns and cross-checks against the oracle. Corrects any deviation > 2%.

Other components: - `src/portfolio.py` — SQLite ORM for all portfolio data (holdings, transactions, snapshots, orders, trailing stops) - `docs/` — GitHub Pages website with auto-refreshed JSON data files - `logs/` — Application logs (rotated at 5 MB, 2 backups) - `src/portfolios.db` — SQLite database (source of truth for all portfolio state)

3.2.2 Automation Layer — `~/openclaw/workspace/`

The automation layer runs as `openclaw-gateway.service` under systemd. It provides the cron engine that drives all scheduled operations, 30+ standalone Python scripts for data gathering and trading, modular agent skills, and the `SOUL.md` personality file.

Scripts (`scripts/`) — 30+ standalone Python scripts that handle everything the LLM should not be trusted to do on its own: data gathering (`morning_data_gather.py` , `afternoon_data_gather.py`), price refresh and dashboard updates (`price_refresh.py`), autonomous trading (`autonomous_trader.py`), decision engine scoring (`decision_engine.py` , 20 signal dimensions), style gate enforcement (`style_gates.py`), AI gate reasoning (`gate_reasoning.py`), weekly candidate discovery (`candidate_screener.py`), style compliance auditing (`style_compliance.py`), trailing stop management (`trailing_stop_manager.py` , `stop_check.py`), ARK tracking (`ark_itk_tracker.py`), and portfolio reconciliation. All scripts source credentials from `~/env_secrets` and log via `bigclaw_logging.py`.

Configuration (`config/`): - `portfolio_universes.json` — Per-portfolio allowed ticker lists (holdings + candidates) - `expert_overrides.json` — Manual conviction score overrides for specific tickers

SOUL.md — The agent's personality, analytical mandate, and behavioral rules. This file is loaded into every OpenClaw agent session and defines who BigClaw is. Key directives include:

- **Prime Directive:** Truth and accuracy above all. Protect the user's financial resources. Objectivity over narrative.
- **Analytical Mandate:** Lead with conviction and directional bias, then explain. Synthesize conflicting signals explicitly. Be bold and interpretive — state clear leans, scenarios with rationale, and thesis application. Minimize raw data dumps; prioritize interpretation.
- **Personality:** Sharp, direct, confident when warranted, brutally honest, forward-looking. Humor auto-adjusts by context (TARS-style calibration: low for alerts, higher for casual chat).
- **Anti-Hallucination:** Never invent prices, percentages, or news. Always call a data tool before responding to any market query. If tools fail, state "data unavailable" — never guess.
- **Autonomy Guardrail:** Never autonomously fix code, edit configs, or debug issues unless explicitly asked. Report problems and wait. This is a hard rule that overrides all other instructions.
- **Fact Verification:** Pull live data via `stock_quote.py` (primary), `technical_analysis.py` (technicals), or web search (fallback) before every analytical response. Cross-verify when possible; flag discrepancies > 2%.

- **Mandatory Skill Usage:** When asked to fact-check, must load and run the `truth-engine` skill (4 independent AI models through a SIFT framework).

Skills — 30 modular analytical capabilities the agent can load on demand. 27 are installed in `~/.openclaw/skills/` (from the ClawHub marketplace) and 3 are custom workspace skills:

Skill	Description
<code>backtest-expert</code>	Expert guidance for systematic backtesting — parameter robustness testing, slippage modeling, bias prevention, interpreting results
<code>finance-news</code>	Market news briefings with AI summaries across US/Europe/Japan markets
<code>financial-planning</code>	Solopreneur business finances — budgeting, cash flow, P&L, projections
<code>fundamental-stock-analysis</code>	Structured scoring playbook — quality, balance sheet, cash flow, valuation, peer ranking
<code>institutional-flow-tracker</code>	Track 13F filings — hedge fund and institutional ownership changes, smart money flow
<code>intellectia-stock-forecast</code>	AI-powered entry/exit analysis, target price predictions, probability calculations
<code>log-analyzer</code>	Parse, search, and analyze application logs; debug from log files; error pattern analysis
<code>market-environment-analysis</code>	Comprehensive global market analysis — risk-on/off assessment, sector analysis, technical indicators
<code>news-summary</code>	Fetch and summarize news from trusted international RSS feeds (workspace custom)
<code>onchain</code>	Crypto portfolio tracking, market data, CEX history, transaction lookups
<code>openinsider</code>	SEC Form 4 insider trading data — director, CEO, and officer buys/sells
<code>options-strategy-advisor</code>	Options pricing (Black-Scholes), Greeks, strategy P&L simulation, earnings plays
<code>portfolio-manager</code>	Portfolio analysis via Alpaca — allocation, risk metrics, diversification, rebalancing
<code>python-dataviz</code>	Professional data visualization with matplotlib, seaborn, plotly — static and interactive
<code>realtime-x-sentiment-tracker</code>	Real-time X/Twitter sentiment polling every 2 hours, alerts on shifts > 5%
<code>sector-analyst</code>	Sector and industry performance analysis — market cycle assessment, rotation patterns
<code>sec-watcher</code>	Monitor SEC EDGAR filings for 50+ AI/tech companies — 8-K, 10-K, 10-Q, insider transactions
<code>self-improve</code>	Agent self-diagnosis — fix configs, prompts, response quality, personality tuning
<code>skill-vetting</code>	Security and utility assessment before installing third-party ClawHub skills
<code>stock-evaluator</code>	Comprehensive stock evaluation — valuation, fundamentals, technicals, entry prices, conviction ratings
<code>stock-strategy-backtester</code>	Backtest trading strategies on historical OHLCV data — win rate, CAGR, drawdown, Sharpe ratio
<code>time-series-analysis</code>	Time series feature engineering, model training, and forecasting
<code>truth-engine</code>	Multi-model fact-checking — 4 independent AI models through SIFT framework with consensus scoring (workspace custom)
<code>uptime-kuma</code>	Interact with Uptime Kuma monitoring server — check status, add/remove monitors
<code>us-market-bubble-detector</code>	Quantitative bubble risk scoring via revised Minsky/Kindleberger framework — VIX, margin debt, breadth, IPO data
<code>yahoo-finance</code>	Stock prices, quotes, fundamentals, earnings, options, dividends, analyst ratings via yfinance

Skill	Description
afrexai-esg-reporting	ESG scoring on Environmental, Social, and Governance factors aligned with 2026 disclosure standards
afrexai-portfolio-risk	Monte Carlo simulations, Value at Risk, stress tests, drawdown, beta, Sharpe ratio
afrexai-tax-planning	Tax-loss harvesting suggestions, entity structure optimization, deduction planning

3.3 Services

Two systemd services run permanently:

Service	Type	Purpose
bigclaw.service	systemd (always-on)	Slack bot + Claude agent + 36 interactive tools
openclaw-gateway	systemd (always-on)	Agent runtime, cron engine, Slack/WhatsApp gateway

A lightweight crontab job (`stop_check.py` every 15 min during market hours) runs outside the LLM loop for zero-token-cost monitoring.

4. Database Schema

The database (SQLite) is deliberately simple and self-contained. Everything that matters is here; nothing is hidden in external services. All portfolio state lives in `src/portfolios.db`.

All scripts connect with WAL (Write-Ahead Logging) journal mode and a 30-second busy timeout. WAL allows concurrent readers and writers without "database is locked" errors — critical because cron jobs (`stop_check`, `price_refresh`) overlap with the autonomous trader during market hours. Every `sqlite3.connect` call across all 15+ scripts enforces `PRAGMA journal_mode=WAL` and `PRAGMA busy_timeout=30000`.

Schema is the committed source of truth (reconciled 2026-07-09). The `CREATE TABLE` statements in `src/portfolio.py init_database()` are authoritative and now match the live `portfolios.db` exactly, so a fresh clone rebuilds a correct database from scratch with no manual `ALTER`s. (Previously `portfolios.purchase_status`, `holdings.target_price/target_set_at/target_source`, and `transactions.order_id/is_correction` existed only in the live DB — a from-scratch rebuild would have crashed on the first query. Found in the Grok codebase review.) All JSON handoff files (`signals.json` and dashboard exports) are written atomically via temp-file + `os.replace` across every writer, including the autonomous trader, so an overlapping reader can never observe a torn file.

4.1 Tables

```
portfolios
  id, name, investment_style, starting_cash, current_cash,
  report_channel, is_active, purchase_status, created_at

holdings
  id, portfolio_id, ticker, shares, avg_cost, rationale,
  first_bought_at, last_bought_at,
  target_price, target_set_at, target_source

transactions
  id, portfolio_id, ticker, action (buy/sell), shares, price,
  total_value, rationale, executed_at, order_id, is_correction
```

```
daily_snapshots
  id, portfolio_id, snapshot_date, total_value, cash,
  holdings_value, daily_return

pending_orders
  id, portfolio_id, ticker, order_type (stop_loss/limit_buy/limit_sell),
  trigger_price, shares, status, triggered_at

trailing_stops
  id, portfolio_id, ticker, high_water_mark, hwm_date, trail_pct,
  trigger_price, status (active/triggered/stale), created_at,
  updated_at, triggered_at
```

5. Portfolio Management Philosophy

BigClaw historically managed up to nine independent paper portfolios. **As of the July 21–22 2026 four-sleeve cutover, four portfolios are live (`is_active=1`); the rest are retained in the DB as inactive history** (see §5.4). Style fidelity remains non-negotiable for the rule-based books. LLM-Commando is the event-driven discretionary stock book (see §5.3).

5.1 Portfolio Definitions

#	Portfolio	Investment Style	Live?	Modeled After
1	Value Picks	Quality Value Investing	Inactive (Jul 2026 cutover)	Buffett / Graham
2	Innovation Fund	Disruptive Innovation	Active	Cathie Wood / ARK
3	Growth Value	Growth at Reasonable Price (GARP)	Inactive	Peter Lynch
4	Income Dividends	Income / Dividend Growth	Inactive	Dividend Aristocrats
5	Momentum Growth	CANSLIM Momentum	Active	William O'Neil
6	Nuclear Renaissance	Nuclear Energy / Domain Expertise	Inactive	Thematic Structural
7	AI Defense & Autonomous	AI Defense / Autonomous Systems	Active	Pentagon Spending Theme
8	LLM-ETF Focus	LLM-Driven ETF-tilted dialectic	Inactive	None — pure model judgment
9	LLM-Commando	LLM-Driven Single-Stock dialectic + radar	Active	None — pure model judgment
—	Treasury Reserve	Cash bucket (historical)	Inactive (retired May 2026)	—

5.2 Portfolio Parameters

Parameter	Value
Starting capital per portfolio	\$100,000 virtual cash

5.3 LLM Discretionary Portfolio (experimental, June 2026 onward)

The 8th and 9th portfolios are deliberate counter-designs to the seven rule-based portfolios. They share the same 3-Sonnet dialectic architecture but differ in candidate universe and prompt preference:

- **LLM-ETF Focus** (formerly named LLM Discretionary, renamed June 10 2026) — original variant, trades whatever the dialectic settles on with no preference. Observed behavior is sector-rotation via SPDR ETFs as the LLM defaults to the most data-rich investable objects (XLK, XLF, XLE, etc.) and rotates between them on macro/news shifts. Curtis observed this and decided to preserve the behavior intact as a real-world test of LLM-driven sector rotation. **Inverse ETFs / short capability (July 2 2026)**. ETF Focus's universe was long-only (sector SPDRs + long factor/macro ETFs), so a bearish regime read could only be expressed as cash or defensive longs — at the time it sat ~86% cash after hawkish-Fed commentary, forecasting a decline it had no way to profit from. Added `INVERSE_ETFS = [SH, PSQ, DOG, RWM]` (-1x S&P/Nasdaq-100/Dow/Russell) to the universe, the market snapshot, and a rendered candidate block. These are ordinary LONG buys that rise when the market falls, so they need no shorting/margin infrastructure and fit the long-only execution + ETF-only mandate. The prompt framing (per Curtis) gates them tightly: cash is the correct call in a genuine UNKNOWN (no directional edge); an inverse ETF is for a CONFIRMED bearish edge only — the SAME VIX 22+ / credit-breadth-deterioration bar used to size down, PLUS a macro driver pushing the tape lower — and explicitly NOT in a low/normal-VIX tape (high-teens VIX is a long-or-cash regime, not a short). Deliberately -1x only; leveraged -2x/-3x inverse are excluded as bad-quadrant volatility-decay risk on the 1-day-to-1-week horizon. Shorts are sized with a target/stop and covered when the down-move plays out — never married. (VIX was low when this shipped; it is a capability for the next down-regime, not a directive to short.)
- **LLM-Commando** (new June 10 2026) — variant with **strict stock-preference language** in all three agent prompts (Bull, Bear, Judge). ETFs are explicitly forbidden except for hedging. Candidate universe expanded to include ~60 curated liquid names from `portfolio_universes.json` (the BigClaw rule-based portfolios' shared universe) so per-ticker Benzinga news flows into the context for individual-stock theses. Dry-run verified: Judge picked AMAT (analyst PT raises) and APA (geopolitical energy thesis) — no ETFs.

The original architecture description below applies to both variants. Differences noted inline.

Common architecture: No style gates, no top-10 rotation, no target-price discipline, no decision-engine scoring. Every trade decision is made by a 3-Sonnet dialectical decision system.

Architecture: 3-Sonnet dialectic. Three Claude Sonnet 4.6 agents run sequentially each weekday at 11:00 CT: 1. **BULL agent** — given the data (portfolio state, sector/factor ETFs, news, journal), builds the strongest case FOR candidate trades. 2. **BEAR agent** — reads the bull's case and same data, builds the strongest case AGAINST. Stage 1 fact-verifies every Bull claim against the data feed; Stage 2 then runs a **mandatory ALREADY-PRICED-IN test** on each surviving thesis — freshness (when did the catalyst go public?), price reaction (has the stock already moved on it? if so, entering is chasing the reaction), and durability (durable or reflexive driver?). A true-but-already-priced thesis is a disqualifying weakness, not a tradeable one (strengthened June 15 2026 after the LLM-Commando AAL miss — bought a day-old oil/MOU catalyst already in the price). 3. **JUDGE agent** — reads data + bull + bear + journal. Its **first move is gap-analysis, not adjudication**: before weighing Bull vs Bear it must name what is absent from BOTH cases that would change the decision (is the catalyst already priced in? durable or reflexive driver? what would make this wrong that neither side named?), emitted in a required `gap_analysis` field. This is the alpha-generating function of the more capable Judge seat (Commando runs Opus 4.8 here vs Sonnet on ETF Focus — an active A/B) — synthesizing two advocates only averages views already in the price; the edge is in the shared omission (added June 15 2026). It must then explicitly address the strongest bear counter-arguments before committing. **Objective = alpha (risk-adjusted return), not raw profit (added June 15 2026)**. The **Commando doctrine** is a commando raid: enter only on a real, not-yet-priced-in edge with a stated favorable reward-to-risk asymmetry. **Exit is governed by conviction, not the clock (revised June 18 2026, replacing the earlier “take the gain, exit fast, don't loiter” timer rule)**: before selling any held position the Judge asks one question — *would I buy this stock right now, at today's price, or do I prefer another opportunity available to me?* If it would still buy, it holds — it does not sell a working trade just to lock a quick gain or because time has passed; if it would not (thesis faded, move exhausted, or a clearly better trade available) it sells and redeploys into the better one. Holding time is irrelevant: quick exits still emerge on their own because a

spent momentum move is one it would no longer buy, but the trigger is always conviction, never a timer, and it never churns out of a name it would still buy today. A name just sold is eligible for fresh re-entry if its edge reappears (re-judged on the same metrics — no loyalty, no aversion). The bad quadrant (small reward / high risk) is rejected even if it might close green; a low-risk trade must beat the money-market rate or hold cash. This is short-window style by design, NOT buy-and-hold — a buy-and-hold re-skin of the platform is a separate future style. Produces strict-JSON trade decisions with required `exit_thesis` field (specific gain target / stop loss / time-based exit).

Macro-regime cycle pack (added June 16 2026). `get_market_snapshot` now also pulls $\wedge$ VIX, HYG, LQD, $\wedge$ TNX, and IWM (1-year history) and computes 3-month (63d) and 6-month (126d) returns alongside 1d/5d/30d. A "MACRO REGIME (cycle tells)" block renders into both portfolios' context: VIX level + trend, high-yield-vs-investment-grade credit (HYG vs LQD), the 10-year yield, offense/defense (XLY vs XLP), breadth (IWM vs SPY), and 3-month sector-rotation leaders/laggards. The two portfolios consume it differently. **ETF Focus** runs a "MACRO REGIME READ" mandate — use the regime block to set risk-on/risk-off aggression and sector tilt for the days ahead, getting in early on leadership that is starting to rotate rather than chasing what already ran. **Commando** runs a "VOLATILITY REGIME" mandate — use VIX to size aggression: a low/falling VIX is a calmer tape where news-backed setups follow through (size up modestly); a high/spiking VIX means whipsaw (throttle back, size down, favor cash). (Updated June 25 2026: the original June-16 "position sectors months-out" framing for ETF Focus was reverted to short-term — see the June 25 reversion note below.) All free via yfinance (no FRED key).

VIX calibration to an empirical threshold (June 26 2026). The volatility-regime mandates initially had no absolute thresholds (just "low/falling" vs "high/spiking" VIX), and the rendered VIX line led with its 5-day percent change, so the LLM read a VIX of 19-20 (a normal level) as a fear spike and threw the size-down / favor-cash / raise-the-bar mandate. That is the direct cause of Commando sizing its best winner (PODD, +10%) at only ~6% of equity and sitting ~90% cash. The threshold was re-derived from data: over the last 252 trading days $\wedge$ VIX averaged 18.1 (median 17.1, 75th percentile 19.1), and the probability of a >3% SPY drop within 10 days held at the ~11% base rate all the way up to VIX 22, then jumped to ~40% (about 4x) in the 22-25 band and ~50% in 25-30. The VIX 18-20 bucket actually showed the *best* mean forward 5-day SPY return (+1.07%) and the lowest drop odds (5%). So the empirical threshold of concern is **VIX 22**, not the high teens. Both LLM portfolios now carry explicit, data-grounded bands - **<22 NORMAL** (trade and size normally; the high teens are not a warning), **22-25 ELEVATED** (size down modestly, tighten stops, raise the conviction bar - the real ~40%-drop-odds zone), **25-30 HIGH** (defensive, size down meaningfully), **>30 EXTREME** (favor cash; hit only once in the last year) - and the rendered VIX context line now carries the band label inline (e.g. "VIX 18.9 [NORMAL] (last-yr median ~17, 75th pct ~19; concern threshold 22; ...)") so the model sees "normal" rather than an alarming percentage jump. Caveat from the same analysis: VIX often rises *with* a drop rather than cleanly ahead of it (a quarter of >3% drop-days ignited from a VIX under 16.8), so 22+ is a genuine "raise your guard" signal but not a forecast - per-position stops still matter regardless of the VIX level.

Fixing the structural over-caution (June 26 2026). Both LLM books had drifted to ~90% cash with single small positions, and the audit found the cause was systemic, not a single bug: the decision system minimizes *realized losses* but is *blind to opportunity cost*, and every mechanism pushed the same way with no counterweight. Three fixes shipped (the VIX recalibration above was the first; these are the rest):

1. **Cash is not free** (both portfolios). In the current ~4% inflation environment, with zero interest on idle paper cash, sitting in cash is a guaranteed ~4%/year real loss. The Bull, Bear, and Judge now treat cash as a slow certain loss rather than a safe default: the hurdle to deploy is LOW (only beat a negative real return), and holding cash is the EXCEPTION that must be justified, not the resting state. Sitting 80-90% cash for weeks in a flat/up/rotating tape is explicitly named a FAILING posture - the skill the experiment tests is harvesting the rotation, not avoiding every loss.
2. **"Already priced in" retuned to "is there still room to run?"** (both portfolios). The Bear's mandatory ALREADY-PRICED-IN test was near-unsatisfiable for a news-driven book: you only ever see a catalyst *after*

it has moved the stock, so "it already moved" was true of nearly every candidate, and the test rejected almost everything - discarding the entire continuation/drift edge (a stock up 3% on fresh strong news routinely runs further). Retuned: a live catalyst always moves the stock first (that is normal continuation, not a disqualifier); reject only if the move plausibly **OVERSHOT** the catalyst (e.g. a parabolic pop on a minor/procedural item) or the catalyst is genuinely **SPENT**. Added a Bull **DEPLOYMENT MANDATE** symmetric to the Bear's reject-test, so each cycle the Bull must surface the best deployable setup or explicitly state why cash beats every name.

3. **Refusal scorecard - the two-sided learning loop** (Commando). The recursive journal only ever recorded trades it *took* (closed positions via the reconciler), so avoided losses *felt* like wins and missed gains were invisible - the journal could only ever teach *more* caution (the "27 consecutive cycles, passivity is my edge" prior was built on just 2 losing trades, with zero record of the winners it refused). Commando now logs every candidate seen-but-not-bought with its price (`data/llm_comando_passed.jsonl`) and renders a "REFUSAL SCORECARD" into each cycle's context: of the names passed on over the last ~7 days, how many rose, the median move, and the biggest misses - the opportunity-cost feedback the journal structurally lacked, so the LLM can finally discover when its bar is too high. ETF Focus sees its full ~21-ETF candidate universe (and their returns) in context every cycle already, so the tracker is lower-value there; only fixes 1-2 apply to it.

The unifying principle: the system was a competent down-market machine (minimize losses) but a poor sideways/up-market machine (capture rotation), because loss-avoidance had no opportunity-cost counter-pressure. These changes add that counter-pressure without re-introducing price-momentum bandwagon-chasing (every entry still needs a citable, still-playing-out catalyst).

Curated LESSONS block + extension/RSI in the compete view (July 17 2026; lessons 5–6 July 22 2026). An evidence-based diagnostic of BigClaw's own trades (415 real buys, 2026-04 → 07) found the single strongest predictor of poor forward return is how *extended* a name is at entry — buys far above their 200-day MA or overbought (RSI>70) mean-revert, the most-extended quintile returning ~-8% vs SPY. A shared `scripts/llm_lessons.py` module now injects a curated **LESSONS LEARNED** block into Commando's data feed every cycle (Bull, Bear, and Judge all read it): (1) extension is risk, not confirmation; (2) buying strength is not an edge; (3) opportunity cost is the exit; (4) a live catalyst is the edge, price alone is a bandwagon; **(5) threatened tariffs / political headlines are not enacted policy** — prefer real reported numbers; only in-force policy (or company already booking the cost) can veto a clean earnings thesis (07-21 lesson: Judge skipped a real beat+raise on Canada-tariff *threat* framing); **(6) day-2 chase after the run is not edge** — rehash of yesterday's earnings after a multi-day pop with no new fact = stand down. These are **information, not hard rules** — the dialectic still decides. Commando flags overbought/extended names in the compete view with `△extended`. Soft LLM governance vs hard entry-gate for rule books (\$6.5b).

Commando event radar + price-is-news (July 21–22 2026). Waiting only for 09:00 / 11:30 / 14:30 CT dialectic sessions was too slow for day-trader style. `scripts/llm_comando_radar.py` polls **every 2 minutes** during RTH (`*/2 8-15 * * 1-5`), reads new Alpaca news since a cursor, and runs a cheap Sonnet **GO** decision without full Bull→Bear→Judge when there is something to act on. Shared daily fire budget: `MAX_EVENT_FIRES_PER_DAY = 100` in `llm_comando_news.py` (~\$0.02–0.03/fire). **price_is_news (holdings fade):** multi-day stock loss with *no* fresh headline is itself an event — if SPY session is not down and the sector ETF session is not down, and the name lags its sector over ~3d, radar wakes research and defaults to **sell half** (system clamps to 50%; residual keeps trailing stops). Market/sector weakness alone is never a sell reason. Cooldown ~20h per ticker; fade scan throttled to ~15 minutes.

Cold-trader doctrine (July 22 2026). Operator laws in Bull/Bear/Judge + radar NORTH STAR: - **REAL > THEATER** — earnings / formal guidance / signed deals beat threatened tariffs; only **ENACTED** (in force) policy can veto a print. - **NO CHASE** — same-session reaction to a live print can still have room; day-2+ rehash after multi-day run with no new fact → stand down. - **COLD EDGE (not passivity)** — constraints remove human

FOMO / threat-fear; they do **not** mean hug cash or trade less. Selective aggression: early on real facts, absent when late and crowded.

(OpenRouter shadow dialectic / Grok-as-Judge is experimental shadow-only and is not documented as production until graduated — live Judge remains Anthropic Opus 4.8 with Sonnet Bull/Bear.)

Hold scorecard - the third learning signal (June 26 2026). The learning loop now covers all three decision types. Buy/sell calls are scored by the reconciler (closed positions vs `exit_thesis`); no-buy calls by the refusal scorecard (above); and now HOLD calls by a hold scorecard. A hold is a real decision under the conviction-exit doctrine, but it had been invisible - each cycle the LLM "held" by deciding zero sells, with no measure of what the hold cost. Commando now tracks each open position's high-water-mark unrealized % (`data/llm_commando_position_peaks.json`) and renders the GIVE-BACK (peak minus current) into context each cycle, e.g. "PODD held since 06-23: now +8.4%, peaked +12.4% - GIVEN BACK 4.0 pts by holding; would you BUY it here today, or are you riding a winner back down?" It also flags a hold that has gone outright negative. This makes the classic failure - riding a winner back down, or sitting in a loser hoping for recovery - a visible, learnable cost instead of an implicit non-decision. Per Curtis: a hold that loses over time should teach, the same as a bad buy or a costly refusal.

The alpha drive - installing a gas pedal, not just releasing brakes (June 29 2026). First live run with all the above showed the limit: Commando read the refusal scorecard ("74% of names I passed rose"), agreed it was a failure mode, and *still* held 0 trades - rationalizing the evidence rather than acting. Per Curtis: every fix so far had only *removed brakes* (VIX, cash-isn't-free, room-to-run, scorecards); none installed a *drive*. The LLM's implicit objective was still "don't lose," which cash satisfies perfectly, so it defaulted to cash and explained it away. The fix changes what Commando optimizes for: (1) the Judge objective is now explicitly to GENERATE ALPHA - beat SPY and the rule-based bots - and a money-market parking posture that risks nothing is named the surest failure ("a thing whose best move is cash is not an investor"); the peer-return block is framed as a scoreboard it is losing. (2) Cash is reframed from safety to FORFEIT, with the burden of proof flipped onto it - to hold cash the Judge must defend the specific claim "the names available to me are more likely to fall than rise," which on a risk-on day (SPY up, VIX normal, credit risk-on) is hard to defend. (3) The reflection field now forces a benchmark-relative self-assessment ("are you BEATING SPY and the bots? if you are trailing while in cash, your caution is the failure, not the edge") to counter the self-reinforcing "27 cycles, passivity is my edge" prior. (4) Persona: an alpha HUNTER, not a goalie, embarrassed to end a green day in cash, who hunts HARDER when the obvious news-makers are weak - and Curtis's framing: "cash is ammunition, not a fortress; deploy it to plunder the market's cash and grow the war chest." Guardrail preserved: every entry still requires a real, citable, still-playing-out catalyst with room to run - the drive is to find the genuine edge harder, never to manufacture one. First dry-run after the change: Commando went from 0 trades to deploying on a genuine catalyst (GOOGL DJIA-inclusion, mechanical multi-day passive flows) while explicitly noting it was losing to the bots. **Apples-to-apples peer scoreboard (July 7 2026).** The 'losing to the bots' framing relies on the peer-return block, which was comparing Commando's ~1-month track record (since its June 10 inception) against the rule-based peers' since-FEBRUARY cumulative - not comparable. `get_peer_returns` now baselines every peer to Commando's first `daily_snapshots` date and reports return over Commando's own window (same as Commando's cumulative), labeled as such. It materially changes the picture (e.g. Growth Value +3.9% since Jun 10 vs -2.8% since Feb; Momentum -6.6% vs -1.7%), so the scoreboard is honest rather than flattering or unfair. **Immediate post-trade trailing-stop init (same day).** The LLM cycles relied on the every-15-min `stop_check` cron to create trailing stops, so a position bought between runs was briefly without a `trailing_stops` row and got flagged UNPROTECTED by the refresh data-health-check (DOCN, bought 9:11, stop created 9:15). Both cycles now call `initialize_stops()` immediately after a filled buy (mirrors the rule-based trader's RULE 5.25). The position was never truly exposed - its `exit_conditions` stop is live from purchase via the 5-min mechanical exit - only the false alert is fixed.

Commando momentum sourcing (added June 17 2026, REMOVED June 25 2026). A

`discover_momentum_leaders()` view briefly fed Commando relative-strength / breakout names from the rule-based screened universe (`signals.json`) alongside the news-makers, with a Judge "MOMENTUM SOURCING"

mandate that treated a clean uptrend as a tradeable thesis even without a headline. **Reverted June 25 2026 per Curtis:** price momentum with no news behind it is bandwagon-chasing, not a thesis. Commando is back to news-driven discovery only — every entry must rest on a citable, still-playing-out catalyst; when the news set is thin, holding cash is the correct call rather than manufacturing a momentum trade. The VIX VOLATILITY REGIME mandate stays (it sizes aggression) but was retuned to drop the "ride the leaders" language.

LLM-portfolio horizon reset (June 25 2026). After a week of observation, two June-16-to-18 changes were reverted to restore the original experiment. (1) **ETF Focus back to SHORT-TERM.** The June-16 pivot had reframed ETF Focus as a multi-month thesis book (wide 25%/15% exits, thesis-break-only triggers, 30-min watcher, "hold through noise"). A longer horizon arguably suits an ETF book, but the experiment *learns faster* with short-term turnover, and the paired A/B only holds if both LLM portfolios share a horizon. ETF Focus is again a short-term, fast-turnover book: tight exits, intraday triggers, quick-profit/cut-loss doctrine, 5-min watcher — same horizon as Commando, differing only on instrument (ETFs vs single stocks). The good post-pivot additions were KEPT: the gap-analysis Judge mandate (retuned to a days-to-week lens), the unified cycle framing, the do-not-open-into-a-binary-event rule, and the macro-regime pack. (2) **Commando back to news-only discovery** (see the paragraph above).

Judge model A/B — Fable 5 vs Opus 4.8 (concluded July 11 2026). A live A/B ran Fable 5 as the Commando Judge (executing) with Opus 4.8 shadowing the identical Bull/Bear input. It concluded on **reliability, not decision quality:** Fable's long adaptive-thinking Judge requests get dropped / time out (`APITimeoutError`), and a timed-out Judge kills the entire cycle — no decisions, no trades. Record: one ~723s near-miss (July 9), then **two of three cycles FATAL on July 10** (morning + midday). Opus shadow ran ~110s with valid JSON every time. Reverted the live Judge to Opus 4.8 (Bull/Bear unchanged, Sonnet 4.6); the shadow A/B block was disabled. **Design lesson:** for a live trading Judge, reliability — completing within the cycle's time budget before the market close — *gates* decision quality, and must be tested under the real timeout, not just dry-runs. The Bull/Bear -> Opus promotion (held until the Judge was settled) is now unblocked as a separate future A/B.

Every-session re-verification + time_exit retired (July 13 2026). Commando now surfaces each holding's persisted ENTRY THESIS + original target + entry date into the Judge's compete view every session (the "WHY BOUGHT" anchor; the data was already stored in holdings.rationale/target_price, just not shown). Doctrine added: the entry thesis is a TIME-STAMPED HYPOTHESIS re-verified every session and classified STRENGTHENED / INTACT / WEAKENED / SPENT - never a standing justification. A few-hour catalyst cannot justify a multi-day hold; the position must be re-justified by CURRENT evidence, the entry thesis serving only as the yardstick for stronger-or-weaker-than-at-entry. Every stock earns its existence every session; holding period is an OUTPUT, not a target. The mechanical time_exit (clock-based forced sale) was RETIRED from evaluate_triggers (shared by watcher + reconciler) - it force-sold VOYA on a timer July 13 despite a fresh same-day KBW raise, contradicting the conviction-not-clock doctrine. stop_hit (risk rail) and target_hit (bank a planned gain between cycles, re-verified next cycle) are KEPT. Dry-run validated: the Judge re-verified all four holdings per-doctrine (MPC strengthening, VOYA intact, RJF 4-days-stale/watch-rotation, THC original-catalyst-stale-but-WF-raise-today-resets-the-clock). Pending: feed this-session mechanical exits into the Judge + a stop-loss reconciliation lens.

Evidence-grounded dialectic (July 14 2026). Every SELL is now classified by REASONING not P&L: thesis_wrong (buy premise proved false - the learning; carries an exit_lesson), thesis_changed (rotation/banked-gain/regime-shift - NOT a negative), or thesis_played_out (predicted move happened - a win). A stop-out is thesis_wrong only if the premise was flawed. The classification is surfaced inline in the journal all three agents read. Bull mines past PLAYED-OUT sells for analogs to strengthen a buy; Bear mines past REASONING-ERROR sells for analogs to bolster an avoid/sell; the Judge classifies every sell (and unclassified mechanical stop-outs it reviews) and REFEREES - weighting track-record-backed arguments over assertions and flagging cherry-picking. Per Curtis: the learning lives in thesis_wrong; thesis_changed is never counted against anything.

ETF Focus event-risk rule (added June 18 2026). Before OPENING a new position, the ETF Focus Judge checks for a binary event (FOMC / CPI / jobs / earnings) in the next 1-2 days and does not establish fresh exposure into it — it waits for the event to clear. Existing positions are still held through events; only opening-into is barred.

Added after a June 17 same-day whipsaw: ETF Focus bought XLF hours before an FOMC decision and had to sell it after the hawkish pivot (Commando's gap-analysis had caught the event risk; ETF noted the meeting but bought anyway).

Agent token ceilings (raised June 17 2026). The June 16 doctrine additions (the `gap_analysis` field, cycle-positioning / regime reasoning, the mandatory Bear priced-in test) made agent outputs longer and began truncating them — on June 17 the ETF Focus Judge overflowed a 4000-token cap and its JSON failed to parse, costing that cycle's deployment. Ceilings were raised for output room with no logic change: ETF Judge 4000→8000 (matching Commando), and the Bull/Bear debate caps 3000/4000→6000.

Recursive learning via journal. Every cycle appends to `data/llm_journal.jsonl` — input snapshot, bull case, bear case, judge decision, executed trades, and (filled in later by Python reconciler) realized P&L vs the predicted `exit_thesis`. Each subsequent cycle reads this journal as input, so the model can see its own track record of wins, losses, and prediction accuracy. Over weeks, the model's stated `patterns_noted` field accumulates as a self-written strategy guide.

Anti-cheating constraints (mechanical, enforced in Python): - Ticker validation: every proposed trade ticker is verified against Alpaca's tradable assets list before submit. - Cash wall: cannot spend more than `current_cash`; enforced by `record_trade`. - Market-hours gate: trades only submitted when Alpaca clock reports market open. - Hallucination prevention via prompt: training cutoff is January 2026; every factual claim must cite the data feed; no inventing news, earnings, or events.

Safety rail — catastrophic drawdown freeze. If total portfolio value drops below \$50,000 (50% of starting capital), `LLM_PORTFOLIO_DRAWDOWN_FREEZE.flag` is written, trading pauses, and a Slack alert fires. Manual resolution required to resume.

Cost. LLM-ETF Focus daily cycle ≈ \$0.13 (three Sonnet 4.6 calls, smaller news context). LLM-Commando daily cycle ≈ \$0.18 (same architecture but expanded ~60-ticker candidate universe inflates input tokens). Combined annual cost: ≈ \$80 for ~252 trading days × 2 portfolios.

Trade closure reconciler (added June 8 2026). A second script `llm_portfolio_reconciler.py` runs daily at 14:50 CT (just before market close — moved from 15:30 CT on June 30 2026, see fix below) to verify each open position against its stated `exit_thesis`. Logic: in priority order, check `stop > target > time_exit`; if any trigger fires, submit the SELL via the canonical Alpaca + `record_trade` path. The Judge prompt now emits a structured `exit_conditions: {target_pct, stop_pct, time_exit_date}` alongside the prose `exit_thesis`; for legacy trades lacking the structured field, a small Sonnet 4.6 call (~\$0.001) extracts the conditions on first encounter. Each closure writes an `outcome` record to `data/llm_outcomes.jsonl` capturing entry/exit prices, realized %, days held, which trigger fired, and whether the original prediction was correct. **This outcomes log is what makes recursive learning real** — without verified prediction outcomes, the daily Judge has no signal to learn from. Periodic synthesis (monthly, to be built ~July 1) will compress accumulated outcomes into a `llm_strategy_guide.md` that the daily prompt reads at top-of-context.

Phantom-closure bug + timing fix (June 30 2026). Two defects let a fired exit silently strand a position and corrupt the learning log. (1) The reconciler ran at 15:30 CT — 30 min AFTER the 16:00 ET close — and submitted a `TimeInForce.DAY` order, which cancels unfilled after hours (despite a stale code comment claiming orders "queue to next open"). So the sell filled 0 shares. (2) `main()` then wrote the closure `outcome` regardless of fill: with `filled_price` 0 it recorded `exit_price 0 -> realized_pct -100%`, while `record_trade` correctly refused the zero-share DB update — leaving the position still held but logged as closed. The next run then skipped it forever as an "already in outcomes but still held" anomaly. Commando's PODD (entry 06-23, target_hit 06-26) was orphaned this way for 4 days, riding +12% back down while the dashboard showed negative upside; ETF Focus's MTUM (06-15) left the same corrupt -100% record. Fix: (a) market-closed guard in `execute_close` — never submit a futile order when `clock.is_open` is false; (b) an integrity gate in `main()` — a closure outcome is

recorded ONLY if the sell actually filled (`filled_qty > 0`); otherwise the position is left OPEN and retried, never fabricated as a -100% phantom; (c) both LLM reconciler crons moved to 14:50 CT so their sells fill during market hours. The two phantom records were purged (backed up) and PODD was exited cleanly (+5.19%, `time_exit`). FOLLOW-UP for the give-back concern: mechanical target/stop exits still only fire once/day at 14:50; moving them into the 5-min watcher would book gains at target-hit intraday ("take the money and run") instead of waiting for the daily sweep. SHIPPED same day (June 30 2026): `check_mechanical_exits()` now runs in BOTH watchers every 5 minutes during market hours, evaluating each holding's target/stop/time exit via the shared reconciler logic (`evaluate_triggers` + the new `build_outcome/compute_predicted_correctly` helpers) and selling through the watcher's clamped, short-safe path (`clamp_sell_to_long`) with the same fill-integrity gate (a closure is recorded only on a real fill). The reconciler stays as the after-close backstop; both write to the same outcomes log (deduped via `origin_date+ticker`) so neither double-closes, and each outcome now carries a `source` field (watcher|reconciler). This is the true fix for the PODD give-back: a target hit at 10am is booked at 10am, not at the 14:50 sweep. **Overnight/wartime risk weighting (July 8 2026)**. The exit-by-conviction rule (which stopped Commando churning to cash) had an unintended edge: it labeled ALL trimming of a winner 'the churn error', so Commando held green names overnight into a war tape and gave the gains back (July 7: the Judge overrode the Bear's 'protect gains, arrive tomorrow with optionality' as churn, even after flagging the overnight risk itself; VRDN peaked +8.5%, exited +6.5%). Per Curtis: a BANKED gain has control/finality/loss-prevention; an unrealized gain is a loan the market can call back overnight; holding past the close buys nothing but gap exposure. Fix (pure doctrine, commit 45ab7d819): the conviction test gains a second half for a position on a real gain - weigh the overnight risk vs banking now and re-entering next session (sell-high/rebuy-low is NOT churn). The weight is modest in a calm tape and rises with war signals: military escalation in the news feed (Iran/Hormuz/tankers), oil trending up (USO), and VIX TREND/rate-of-change even under 22 (absolute VIX level dials INTRADAY aggression, TREND dials the OVERNIGHT-hold decision - separate dials). Give the Bear more weight in wartime. It is a rebalance of when-to-hold, NOT a retreat to cash (still deploy intraday, re-enter next session), so it preserves the alpha drive. The signals were already in the Judge's context; only the doctrine changed.

Dashboard planned-actions made real — single source of truth (June 30 2026). The website's "planned actions" panel was built by `build_planned_actions.py`, a *separate reimplement*ation of the trader's logic that had drifted: it applied a blanket "sell anything held that is not in the top 10 by score" to every portfolio, ignoring (a) target-price discipline (Innovation Fund holds to its analyst price target, not the top-10 rule), (b) the style-gate filter on candidates, and (c) the per-portfolio universe. Result: it showed **12 planned sells when the trader would actually make 1** — e.g. 8 Innovation Fund names held under discipline, plus names pushed out of the top 10 by candidates the trader would never buy. Curtis: "the dashboard needs to reflect what's real, not a phantom plan." Fix: the trader's per-portfolio planning block was extracted into a module-level `plan_portfolio()` in `autonomous_trader.py` (returns to `sell/to_buy/target`, applying the universe + style gate + target-price discipline — pure planning, no execution), and `build_planned_actions.py` now CALLS that function instead of re-deriving the plan. The dashboard panel is therefore exactly what the trader will do. Buys are still sized cash-aware for display. (Emergency Phase-1 safety sells, `score <= -3`, are rare and not separately projected.)

Intraday trigger watcher (added June 8 2026). Per Curtis: "11 AM was my preconceived bias. The LLM should trade when IT sees opportunity." The morning cycle was moved from 11:00 to 09:00 CT, and the JUDGE prompt now requires an `intraday_triggers` field where the LLM specifies up to 8 watch conditions per day — price-based (e.g., "if NVDA breaks \$210"), news-based (e.g., keyword list like ["Fed", "Powell", "FOMC", "rate decision"]), or time-based (e.g., final-hour check). A new script `scripts/llm_portfolio_watcher.py` runs every 5 minutes during market hours (cron `* /5 8-15 * * 1-5`), reads the morning's pending triggers from `data/llm_pending_triggers.json`, polls yfinance for price triggers, fetches CNBC RSS + Reuters via Google News for news triggers, and checks time triggers against the clock. When any trigger matches, a focused single-Sonnet LLM call (no Bull/Bear dialectic — the thesis was already formed this morning) decides: execute the original `action_intent` as planned, modify it, or stand down. Decisions execute via the canonical Alpaca + `record_trade` pipeline. Daily budget cap: max 6 fires per day across all triggers, tracked in the state file's `fires_today` counter. Each fire costs ~\$0.05; worst case daily total is \$0.13 morning + 6 × \$0.05 = \$0.43 (\$110/year). Each fire appends

a `trigger_response` entry to the journal and an "Intraday Trigger Fire" section to today's `data/llm_decisions/YYYY-MM-DD.md`, so Curtis can browse exactly what triggered, how the LLM responded, and what executed.

Watcher execution-aware retry + fill-failure alert (added June 25 2026). A decided intraday trade was being treated as done based on the LLM's *decision*, not on whether it *filled*. On June 23 2026, as the market sold off at the open, the Commando watcher's protective SELL of LRCX was submitted as a market order ~1 minute after the open, did not fill within the 30s window, and was canceled (`filled 0/15`) — yet the trigger was marked `consumed` and a fire counted, so the exit was silently abandoned and the position stayed exposed (the 09:00 deliberative cycle independently sold it 33 minutes later; books stayed reconciled — no phantom shares, no short). Fix, applied to **both** watchers: (1) a submitted-but-underfilled trade keeps its trigger in a new `retrying` status and is re-attempted on later polls — re-running the focused LLM call, which re-validates conviction and re-clamps to the live Alpaca long via `clamp_sell_to_long`, so a retry can never oversell or open a short — capped at `EXEC_RETRY_CAP=3` attempts, after which the trigger is `consumed` and flagged `exec_failed`; (2) any decided trade that does not fill now raises a loud 🚨 Slack alert ("position STILL HELD, retrying / MANUAL ACTION NEEDED") instead of the previous buried (`0/N` trades) line. `cleanup_obsolete_triggers` was extended to also clear `retrying` triggers whose position has since been closed by another path. Auto-retry reliably covers price-stop exits (the condition persists and re-fires); news/time-trigger fill failures rely on the alert plus the next scheduled cycle. Note the ETF Focus watcher polls every 30 min vs Commando's 5 min, so its retry cadence is slower.

Experimental purpose. The rule-based sleeves are the control. Commando is the treatment (event-driven + dialectic). Direct comparison: does LLM judgment with structured self-feedback and radar beat hand-coded rules and SPY?

5.4 Four-Sleeve Live Book (July 21–22 2026)

Goal: concentrate labor and capital on books that can generate alpha with lower ops load; cut thematic/duplicate sleeves that mostly delivered sector beta.

Live sleeve	Mode	Notes
Innovation Fund	Rule IPS + decision engine	Active
Momentum Growth	Rule IPS + decision engine	Active
AI Defense & Autonomous	Rule IPS + decision engine	Active
LLM-Commando	Dialectic + radar + watcher	Renamed display path LLM-Commando (<code>portfolio_id=10</code>); script filenames still <code>llm_comando_*</code> for path safety

Inactive (DB retained, `is_active=0`): Value Picks, Growth Value, Income Dividends, Nuclear Renaissance, LLM-ETF Focus, Treasury Reserve. Killed books were flattened at/near market close where possible; residual paper positions that Alpaca marks **inactive/untradable** (e.g. CPRX) may remain on the broker account only — BigClaw DB is cleared for those names; guards treat Alpaca orphans as **warn-only**, not a global halt.

Website / charts: performance and holdings views filter `is_active=1` only (`generate_chart.py`, dashboard straighten). See `data/FOUR_SLEEVE_README.txt` on the Pi. | Maximum holdings per portfolio | 10 | | Minimum holdings per portfolio | 7 (triggers swap/add if breached) | | Maximum single position | 20% of total portfolio value (holdings + cash) | | Rebalance trim target | 18% of total portfolio value | | Concentration monitoring | Every 2 hours (Slack warning if > 20%) | | Auto-rebalance | First trading day of each month | | Cash reserve | Minimum 2% of starting capital retained | | Maximum single order | \$25,000 |

Parameter constraints and expert overrides are defined in configuration files so future maintainers can expand or tighten them without touching core code.

Each portfolio operates within a curated universe of allowed tickers stored in `config/portfolio_universes.json`. The universe is split into two tiers: **holdings** (tickers the portfolio currently owns) and **candidates** (pre-approved tickers eligible for swap-in when the decision engine identifies a stronger alternative). Specific tickers change over time as the engine recommends swaps and the market evolves — the configuration file is the authoritative, living source for current membership.

Value Picks — The universe centers on blue-chip stalwarts that Benjamin Graham would recognize: mega-cap consumer staples, financials with fortress balance sheets, healthcare giants, and legacy industrials trading below intrinsic value. These are companies with decades of earnings history, wide moats, and the kind of boring predictability that deep-value investors prize. The decision engine heavily weights P/E, debt-to-equity, and insider activity for this portfolio.

Innovation Fund — This is the widest universe, deliberately so. It spans disruptive technology across AI, quantum computing, biotech, fintech, cloud infrastructure, and high-growth SaaS — companies that are redefining industries or creating entirely new ones. Many of these names are volatile and unprofitable by traditional metrics, which is exactly why the engine zeroes out P/E and debt signals for this style and instead emphasizes revenue growth, relative strength, and momentum crossovers.

Growth Value — A tightly curated universe of large-cap compounders that deliver strong earnings growth at reasonable valuations — the "best of both worlds" names that Peter Lynch built his career on. These are household-name technology and financial companies that dominate their markets but still have room to grow. The engine balances technical and fundamental signals roughly equally for this style.

Income Dividends — REITs, utilities, MLPs, consumer staples, and dividend aristocrats — companies selected primarily for reliable, growing income streams. The universe favors names with long dividend track records, sustainable payout ratios, and defensive characteristics during market downturns. Bond market signals and dividend yield carry double weight in this portfolio's scoring.

Momentum Growth — The universe is sector-agnostic and changes the most frequently. It tracks names showing the strongest relative strength, price acceleration, and technical breakouts — wherever they occur. The engine applies its heaviest momentum weighting here (MACD, SMA crossovers, and relative strength all at 2×) and largely ignores fundamentals. This portfolio has the tightest trailing stop (10%) because momentum names reverse fast.

Nuclear Renaissance — A focused thematic universe covering the nuclear energy supply chain: uranium miners, fuel processors, reactor designers (both conventional and SMR), and nuclear-adjacent power generators. This is a structural thesis — the multi-decade buildout of nuclear capacity for AI data centers and grid decarbonization — so the universe is deliberately small and deep rather than broad.

AI Defense & Autonomous — Defense primes, autonomous systems developers, drone manufacturers, space companies, and cybersecurity firms positioned to capture the Pentagon's accelerating spending on AI-enabled warfare and autonomous platforms. The universe includes both established contractors and emerging pure-plays. Revenue growth carries double weight here, reflecting the sector's rapid budget expansion.

Universe files can be updated without touching core code. The decision engine's `--rescreen` mode evaluates all candidates in each universe and recommends swaps when a candidate outscores the weakest current holding by 3 or more points.

Treasury Reserve (retired May 1, 2026). An eighth portfolio held \$300K of SGOV as a money-market reserve. It was retired after per-portfolio cash walls (full reset April 10) made a separate cash-bucket portfolio unnecessary,

and after Alpaca paper trading was confirmed not to credit dividends — making the SGOV strategy unable to validate in paper mode. The portfolio row remains in the DB with `is_active=0` for historical preservation; no transactions are accepted while inactive. All website and report logic filters by `is_active=1` so retired portfolios are invisible.

6. Decision Engine & Scoring System

The analytical heart of BigClaw is `decision_engine.py`. It evaluates every candidate ticker across 20 signal dimensions (technical, fundamental, quality, sentiment, macro, and override factors) and then multiplies those raw signals by a style-specific weight matrix (0 = ignore, 1 = normal, 2 = double emphasis). The result is a composite score that is mapped to clear action labels.

Before a ticker even reaches the scoring engine, it must pass through **style gate checks** (see Section 6.5) — hard pre-buy filters that ensure only eligible stocks enter a portfolio's scoring pipeline. Gates are the bouncer; weights are the judge.

This weighted, multi-factor approach is why BigClaw stays true to each portfolio's thesis instead of chasing the same momentum names across every style.

6.1 Signal Dimensions

#	Category	What It Measures	Score Range
1	RSI	Relative Strength Index (14-day)	-1 to +1
2	MACD	MACD/signal line crossovers	-0.5 to +1
3	SMA50	Price vs 50-day simple moving average	-0.5 to +1
4	SMA200	Price vs 200-day simple moving average	-0.5 to +1
5	Cross	Golden cross (50 > 200) or death cross	-0.5 to +1
6	RelStrength	1-month return vs sector ETF	-0.5 to +1
7	EarningsGrowth	Quarterly earnings growth rate	-1 to +1
8	RevenueGrowth	Revenue growth rate	-1 to 0
9	PE	Forward P/E vs trailing P/E direction	-1 to 0
10	DebtEquity	Debt-to-equity ratio (flag if > 100)	-1 to 0
11	ShortInterest	Short float percentage (flag if > 10%)	-1 to 0
12	Insider	Net insider buying vs selling (20 transactions)	-1 to +1
13	BondMkt	Bond market signal (fed funds, yield curve)	Variable
14	ValueOverride	Analyst target discount, RSI oversold, P/B < 1, capitulation volume, expert overrides	0 to +5
15	DividendYield	Dividend yield level	-1 to +1
16	PEG	P/E ÷ earnings growth (Lynch's key metric). Uses best of forward/trailing PEG.	-1 to +1
17	ROE	Return on equity (Buffett's #1). ≥20% excellent, ≥15% good, <10% weak.	-1 to +1
18	FCF	Free cash flow positive/negative + FCF yield context if >5%.	-1 to +1
19	GrossMargin	Gross margin width: ≥40% wide moat, ≥30% decent, <30% thin.	-1 to +1
20	PayoutSafety	Dividend payout ratio: <60% safe, <80% moderate, >80% stretched. Skips non-dividend stocks.	-1 to +1

Signals 1–6 are **Technical**, 7–10 are **Fundamental**, 16–20 are **Quality Fundamentals** (new), and 11–15 are **Sentiment / Macro / Override**.

6.2 Style-Specific Signal Weights

Each portfolio multiplies signal categories differently. Weight 0 = ignore, 1 = normal, 2 = double emphasis. The 5 new quality dimensions (PEG, ROE, FCF, GrossMargin, PayoutSafety) allow portfolios to differentiate on fundamental quality rather than relying solely on price-action and binary earnings signals.

Signal	Value	Innovation	Growth	Income	Momentum	Nuclear	Defense
RSI	0.5	0.5	0.5	0.5	1.5	1	1
MACD	0	1	0.5	0	2	1	1
SMA50	0	1	0.5	0.5	2	1	1
SMA200	0.5	0.5	1	0.5	1.5	1	1
Cross	0	1	0.5	0	2	1	1
RelStrength	0	1.5	1	0	2	1.5	1.5
EarningsGrowth	1	0.5	1	1	1.5	1	1
RevenueGrowth	0.5	2	1	0.5	0.5	1.5	2
PE	1.5	0	0.5	1	0	0.5	0.5
DebtEquity	1.5	0.5	1	1.5	0	1	1
ShortInterest	0.5	0.5	1	0.5	1	1	1
Insider	1.5	0.5	1.5	1	0.5	1.5	1.5
BondMkt	0.5	0.5	0.5	2	0	0.5	0.5
ValueOverride	2	0	1	1	0	1	0.5
DividendYield	1	0	0.5	2	0	0	0
PEG	0.5	0	2	0	0	0	0
ROE	2	0	1	0.5	1	0.5	0.5
FCF	2	0	1	1.5	0	1	0.5
GrossMargin	1.5	0	0.5	0	0	0	0
PayoutSafety	0	0	0	2	0	0	0

Key design choices in the weight matrix: - **Value Picks**: ROE=2, FCF=2, GrossMargin=1.5 — quality is the defining characteristic. - **Growth Value**: PEG=2 — Lynch's single most important metric, replaces separate PE + EarningsGrowth dependency. - **Income Dividends**: PayoutSafety=2, BondMkt=2, DividendYield=2 — sustainable income over yield-chasing. - **Momentum Growth**: EarningsGrowth=1.5 — O'Neil requires earnings-confirmed momentum, not pure price. - **Innovation Fund**: All quality signals zeroed — irrelevant for pre-revenue/high-growth disruptors.

6.3 Score-to-Label Mapping

Score	Label	Action
>= 3	STRONG BUY / ADD	Buy candidate
1 to 2	BUY / HOLD	Hold or add on dip
0	HOLD	No action
-1 to -2	WATCH / CAUTION	Monitor closely

Score	Label	Action
<= -3	SELL / TRIM	Trim or exit

6.4 Special Rules

Special rules baked into the engine enforce discipline:

- **Bond headwind override:** If bond market signal <= -2 and ticker score is marginal (1-2) with weak value override, score is forced to 0 (no buy).
- **Value override cap:** If raw technical score <= -5 and value override is < 3, the override is prevented from rescuing the ticker above -3 (still flagged as sell/trim).
- **Rescreen mode:** When run with `--rescreen`, the engine also scores candidate tickers from portfolio universes and generates swap recommendations (candidate must outscore weakest holding by >= 3 points).

6.5 Style Gate Checks (`style_gates.py`)

Style gates are hard pre-buy filters that run **before** the scoring engine. A ticker that fails a gate is blocked from the portfolio regardless of how high it might score. This is the primary mechanism preventing portfolio convergence — it ensures a pure AI company never enters Nuclear Renaissance, and a zero-dividend growth stock never enters Income Dividends.

Three severity levels:

Severity	Meaning	AI Reasoning?
pass	Allowed into scoring pipeline	No
gate	Borderline — close to failing	Yes (escalated to Opus)
reject	Hard block, no exceptions	No

Per-portfolio gate rules:

Portfolio	Gate Checks
Value Picks	P/E < 35, ROE > 10%, positive earnings. Hard reject: P/E > 60.
Growth Value	PEG < 3.0 (best of forward/trailing). Hard reject: P/E > 80.
Income Dividends	Dividend yield ≥ 1.5%. REITs/MLPs/ETFs exempt from payout check.
Innovation Fund	Must connect to 1 of 5 platforms (AI, genomics, fintech, energy, space). Legacy industries rejected.
Momentum Growth	Earnings growth, ROE, institutional ownership gates (O'Neil CANSLIM).
Nuclear Renaissance	Sector gate: only nuclear-related tickers. Blocks 82% of raw utility screens.
AI Defense & Autonomous	Sector gate: defense-related only. Blocks 69% of raw tech screens.

Entity-type awareness: REITs, MLPs, and ETFs are exempt from payout ratio checks. Banks are exempt from FCF checks. Companies with negative book equity (aggressive buybacks like LOW) are exempt from ROE checks.

6.5b Entry Gate — Anti-Extension / Overbought (`autonomous_trader.passes_entry_gate` , July 17 2026)

A mean-reversion / anti-chasing filter applied to **candidates only** (it never force-sells an existing holding) inside `plan_portfolio`, right after the style gate. It is naturally scoped to the 7 rule-based portfolios — the 2 LLM

portfolios never reach `plan_portfolio`, and an unknown portfolio name has no gate and always passes.

Basis. An evidence-based diagnostic of 415 real buys (all portfolios, 2026-04 → 07) ranked every at-entry feature by correlation with realized alpha vs SPY. The strongest predictor was **extension above the 200-day MA (corr -0.30)**, and the whole top of the list — 5-day momentum, 60-day momentum, volatility, extension vs the 20/50-day MAs, entry-day chase — formed one coherent theme (all negative): the further a name had already run at entry, the worse it did. Options-flow signals ranked dead last (noise). A simulated `ext<+10%` gate flipped the 415 buys from **-1.4% to +1.1%** average return and lifted the win rate from 47% to 59%.

Style-aware thresholds (veto a candidate whose extension or RSI exceeds its style's limit; vetoes only on data that is present, so a missing indicator never blocks a buy):

Portfolio	max % above 200d MA	max RSI(14)
Value Picks, Income Dividends	12%	70
Growth Value	15%	72
Innovation, Momentum, Nuclear, AI Defense	25%	80

Value/mean-reversion styles gate tight; momentum/growth styles gate loose so only egregious chases are killed (e.g. MPC at +40% over its 200d MA / RSI 88) without gutting a momentum mandate. Data source:

`decision_engine.score_ticker` now emits raw `rsi14 + pct_above_200ma` per ticker, carried through `format_json` across the subprocess JSON boundary into the trader's `signal_map`. Thresholds live in the `ENTRY_GATE` map in `autonomous_trader.py`. The gate is shared by the live trader and the dashboard planned-actions panel (both call `plan_portfolio`).

Why entry, not exit. A companion audit of 60 real trailing-stop sells found the absolute stops *protect* the rule-based books (stopped-out names fell a further ~4.65% to date on average), so the stops were **kept**, not removed; a trailing-stop sweep further showed a stop *added on top of good entries makes them worse*. The root cause of underperformance is the entry (chasing extension), and best-in-class opportunity-cost rotation already handles the exit (§7). The gate fixes the entry; the exit was left alone.

6.6 IPS Criteria Are Authoritative (`gate_reasoning` removed May 2026)

Style gates are pure deterministic rules from each portfolio's IPS, defined by the April 2026 four-LLM thesis debate. A ticker either passes a gate or fails it; there is no AI override layer.

A previous design had a `gate_reasoning.py` module that escalated borderline failures to Claude Opus 4.6 for a BLOCK/ALLOW decision. That module was retired May 1, 2026 and moved to `scripts/attic/`. Reasons: - It generated ~470 Opus calls per Saturday (~\$28/run, ~\$1,460/year) for what amounted to second-guessing the IPS design - The 20-dimension scoring engine already handles dynamic re-evaluation — when a held stock's metrics deteriorate, its score drops and best-in-class rotation kicks it out - Design principle: if a stock fails today's gate, it fails. If its metrics improve later, it passes the next screen and is re-evaluated for entry. No need for an AI to second-guess each case.

The four-LLM debate produced explicit criteria for each portfolio (Value Picks: $P/E \leq 25 + ROE \geq 15\%$; Income Dividends: 10-year streak + Chowder Rule + $yield \geq 1.5\%$; Nuclear and AI Defense use whitelists). Those criteria are the design. Anything that fails them is rejected; anything that passes is scored by the 20-dim engine and considered for entry by best-in-class rotation.

6.7 Candidate Discovery Screener (`candidate_screener.py`)

The candidate screener is a weekly pipeline that discovers new stocks for each portfolio. It runs every Saturday at 9:00 AM CT and follows this flow:

1. **Screen** — Finviz screener with portfolio-specific filters (2–3 filter sets per portfolio)
2. **Gate** — Deterministic IPS-criteria gate checks filter raw results. Failures are dropped — no AI override.
3. **Update** — Survivors are added to `portfolio_universes.json` as candidates (max 10 per portfolio, matching the holding cap)
4. **Compare** — The daily rescreen (`decision_engine --rescreen`) scores new candidates against holdings
5. **Swap** — If a candidate outcores the weakest holding by ≥ 3 points (`SWAP_THRESHOLD`), the swap fires: sell incumbent, buy challenger

This closes the loop on candidate freshness — the portfolio universes are no longer static hand-curated lists but are refreshed weekly with market-wide discovery.

6.8 Fundamentals Cache (`fundamentals_cache.py`)

Yahoo throttled the rule-based scorer on June 16 2026 (101 "Too Many Requests" responses) because `decision_engine`, `candidate_screener`, and `style_gates` each fetch `yf.Ticker(ticker).info` per candidate every run — 100+ per-ticker scrapes. But `.info` is fundamentals (P/E, margins, ROE, analyst target, sector) that change daily at most.

`scripts/fundamentals_cache.py` interposes a SQLite cache (`data/fundamentals_cache.db`). `get_info(ticker)` serves the cache, refreshing from yfinance at most about once per 20 hours with exponential backoff + jitter, and — critically — **serves the stale cached row if a refresh 429s**, so the scorer degrades gracefully instead of losing data. It is wired into the per-candidate hot paths: `decision_engine`, `candidate_screener`, `style_gates` ($\times 3$), `style_compliance`, and `autonomous_trader` ($\times 2$). Measured: cold fetch ~ 0.5 -3s, warm hit ~ 0.1 s with zero yfinance calls. A pre-warm cron (system crontab) warms the cache at 08:30 CT weekdays, before the 10:00 trade run, so the run hits an all-warm cache. Lower-frequency `.info` sites (`stock_breakdown`, `research_dossier`, `dividend_analyzer`, `macro_scanner`) still call yfinance directly — migrate later if needed.

6.9 Price Data via Alpaca + External-Scraper Resilience (June 25 2026)

The daily rule-based rescreen (`decision_engine.py --rescreen`, ~ 543 tickers) had been timing out, freezing `docs/data/signals.json` and showing day-old scores on the dashboard. Root cause was two unreliable external scrapers in the per-ticker loop. Both are now handled; the full rescreen dropped from ~ 44 min (and rising) to **~ 2.5 min**, comfortably under the 30-min subprocess timeout.

Prices moved from yfinance to Alpaca. A new `get_daily_bars(tickers, start, end)` in `src/alpaca_data.py` fetches daily split/dividend-adjusted OHLCV bars from Alpaca and returns a DataFrame shaped exactly like `yf.download()` (MultiIndex columns `Field x Ticker`), so it is a drop-in for the engine's bulk price fetch. 554 tickers fetch in ~ 70 s with no rate-limit, versus yfinance throttling hard (1811 "Too Many Requests" in one morning, even on SPY/QQQ) and the bulk download timing out. Symbol quirks (`BRK-B` $\rightarrow$ `BRK.B`) and ADRs are handled. There are two fallbacks: any individual ticker Alpaca cannot serve falls back to yfinance inside `get_daily_bars`, and if Alpaca returns nothing at all the engine falls back to the original yfinance bulk download. SPY history (used for the relative-strength signal) is now fetched **once** per rescreen and passed into `analyze_falling_knife`, instead of being re-fetched per-ticker (543x). The yfinance earnings-calendar lookup was made fail-fast (`attempts=1`) so a yfinance throttle can no longer block the rescreen. **Data-stream pre-flight (July 3 2026).** yfinance is intermittently unreliable on BULK multi-symbol downloads (a single-symbol fetch can succeed while the ~ 30 -symbol $\times 1$ y bulk call returns all-NaN). On July 3 the bulk call NaN-d and both LLM portfolios ran a cycle on a BLANK market snapshot (`get_market_snapshot` still uses `yf.download` with no fallback;

the decision engine was migrated to Alpaca but the LLM books were not). scripts/data_stream_preflight.py (cron 8:15 CT weekdays, before the 9:00 refresh and 10:00 trade) checks the four streams trading depends on - Alpaca trading API, Alpaca market data (get_daily_bars), Alpaca news, and a REALISTIC bulk yfinance download - and posts a GREEN/CAUTION/RED light to Slack + writes data/data_preflight_status.json. It DETECTS a dead feed at the door; the paired CURE (still pending) is to point the LLM get_market_snapshot at Alpaca get_daily_bars with a yfinance fallback + a blind-data guard so a cycle never trades on a blank snapshot. **Alpaca paper-account reset + pre-trade guard (July 7 2026)**. Alpaca silently DELETED equity positions from many paper accounts (confirmed widespread on their community forum) with no orders, fills, journals, or account-activities. BigClaw lost ~\$773K of longs; equity went ~\$1.02M -> ~\$250K with the market flat and only ~\$66K traded (impossible from trading = an account-level wipe). Signature: equity fell by exactly the positions' value with NO cash proceeds. Because the DB still listed the positions, the stop/exit sells filled into the emptied account and opened 6 unintended shorts (clamp_sell_to_long did not catch them on all paths). Alpaca RESTORED the account hours later (reconciled 63/63 tickers, shorts unwound). BigClaw's research was never at risk - the journals/outcomes/patterns live in the DB, not the broker account (the reason a broker wipe is survivable and a platform migration is bounded). FIX (commit 0af712eac): verify_account_synced() in autonomous_trader.py - before any trading, compare live Alpaca positions to the DB; if >=5 DB-held tickers are missing at Alpaca, set the global MISMATCH kill-switch (logs/ALPACA_MISMATCH.flag) + Slack alert and halt, so a future reset auto-freezes BigClaw instead of trading into a phantom book. Wired into ALL 9 order-placing scripts: autonomous_trader, stop_check, trade_executor, and the 3 Commando + 3 ETF-Focus (cycle/watcher/reconciler) - stop_check and trade_executor had never checked the kill-switch before. Tolerates small fill lag; manual clear to resume; fails open on a transient check error.

Atomic dashboard-JSON writes (July 1 2026). docs/data/signals.json was corrupted into a torn file — a complete JSON object followed by ~107 KB of leftover bytes from an older, longer version — which crashed price_refresh with JSONDecodeError: Extra data and failed the whole refresh. Root cause: signals.json has FIVE writers (export_signals.py x2 — rescreen output + holdings enrich; price_refresh.py; build_planned_actions.py; export_dashboard.py) and they are not serialized. The 10:00 autonomous_trader post-trade dashboard export overlapped refresh_all's rescreen, and two plain open("w") writes interleaved into one file. Fix: every signals.json write is now ATOMIC — write to signals.json.tmp.<pid> then os.replace() (atomic on the same filesystem), so a reader always sees a complete file and concurrent writers can at worst last-writer-win (self-heals next cycle), never tear. The corrupt file was repaired by truncating to the valid object (backed up as .corrupt-*). (portfolios.json / market.json have the same multi-writer shape and could get the same treatment if they ever tear.) **Schedule stagger (July 1 2026)**. The underlying overlap was that refresh_all.sh and autonomous_trader.py BOTH fired at 10:00 CT. The pre-trade refresh was moved to 9:40 CT (system crontab) so the full ~15-min pipeline completes (~9:55) before the 10:00 trader reads signals.json — the trader now acts on fresh, complete scores with no write overlap. The trader only needs the export_signals step (first ~4 min), which finishes ~9:44 regardless. Full refresh_all schedule: 9:00, 9:40, 12:00, 14:00, 16:30 CT; trader 10:00. **Intraday price-only refresh (July 9 2026)**. To keep the website's prices and portfolio values fresh between the 5 heavy refresh_all anchors, standalone price_refresh.py (LLM-free, ~1 min, self-commits/pushes portfolios.json / market.json / signals.json) also runs at 10:15, 10:45, 11:15, 11:45, 12:45, 13:15, 13:45, 14:30 CT — scheduled clear of the refresh_all git windows to avoid concurrent-push contention. Net dashboard price freshness is ~30 min through the session; charts and the full dashboard export still refresh only at the 5 anchors. Two further intraday publishes (same day): each intraday slot also runs build_planned_actions.py first, so the forward **planned-actions** list stays current (price_refresh preserves the injected key when it rewrites signals.json); and a push_docs.sh cron at 9:20 / 11:50 / 14:50 CT publishes the **LLM Commando/ETF dashboard panels** right after each dialectic cycle. push_docs.sh is an flock-serialized, retrying commit/push helper, so these extra pushers never collide with refresh_all / price_refresh on the git index lock (they queue rather than fail). **Cash Remaining fix (July 13 2026)**: price_refresh.py omitted the authoritative current-cash field from portfolios.json, so the frontend fell back to startingCash-minus-invested (which ignores realized P&L) and showed a portfolio down on the year as still holding its full starting cash. price_refresh now writes cash (=current_cash) and holdingsValue, matching src/export_dashboard.py, so Cash Remaining reflects the DB truth.

finviz circuit breaker. finviz (the optional short-interest + insider signals) is a flaky scraper with no rate-limit SLA. When it is down it was failing on every ticker with a 2-attempt / 3s-delay retry storm (~592 failures x 3s ~ = 30 min - the dominant cause of the timeout). It is now fail-fast (`attempts=1` , no delay) plus a per-run circuit breaker keyed on the primary signal (`ticker_fundament`): after 12 consecutive failures finviz is disabled for the rest of the run and the remaining tickers skip it entirely. These are minor supplementary signals, so the engine degrades gracefully - scores computed without finviz match the prior baseline to within ~0.8 points mean.

Net effect: the rescreen is now resilient to a yfinance price throttle (Alpaca), a yfinance per-ticker throttle (fail-fast calendar/EPS), and a finviz outage (circuit breaker). The fundamentals cache (6.8) continues to serve yfinance `.info` from SQLite.

7. Autonomous Trading Logic & Execution

Trading is orchestrated once per day at 10:30 AM ET by `autonomous_trader.py` (and reinforced by the intraday stop-check script). The flow is intentionally sell-first: liquidate weak positions and triggered stops before deploying cash into new opportunities. This prevents over-allocation and ensures cash is always available for the best opportunities.

7.1 Execution Flow

1. **Sync** — Reconcile SQLite DB with Alpaca paper account (positions + cash)
2. **Analyze** — Run `decision_engine.py --json --rescreen` to generate signals
3. **Monthly rebalance** — On the first trading day of each month, trim any position exceeding 20% of portfolio value back to 18%
4. **Sells first** — Free cash before buying: - Score <= -5: SELL ALL (100% of position) - Score <= -3: TRIM (50% of position) - Swap recommendation: SELL ALL (replace with stronger candidate)
5. **Trailing stop check** — Execute any triggered trailing stops (between sells and buys)
6. **Buys** — Deploy available cash: - **Style gate check first** — every buy candidate must pass `passes_style_gate()` before execution. Blocked tickers are logged and skipped. - Score >= 3 required to buy (normal mode) - Score >= 2 if portfolio is > 60% cash (deployment mode) - Score >= 1 if part of a swap recommendation
7. **Post-trade sync** — Re-compare DB positions against Alpaca after all trades complete. Only mismatches that persist after execution are reported. This eliminates false alerts from pre-trade discrepancies that the trades themselves resolve.
8. **Summary** — Post execution report to Slack

7.2 Position Sizing

Score	Allocation	Typical Size
>= 5	12% of starting capital	~\$12,000
3 to 4	8% of starting capital	~\$8,000
1 to 2 (swap/deploy)	5% of starting capital	~\$5,000

7.3 Safety Rules

These rules must never be relaxed:

- **Trading window:** Weekdays, market hours only (Alpaca clock). Rule trader ~10:00 AM–4:00 PM ET; Commando radar/watcher during RTH CT windows.

- **Cash reserve:** Minimum 2% of starting capital retained per portfolio (rule books)
- **Max single order:** \$25,000 (rule books)
- **Round-trip limit:** Max 3 round-trips per ticker per week (rule books; prevents churning)
- **Max holdings:** 10 positions per portfolio (rule books)
- **Min holdings:** 7 positions per portfolio (rule books; triggers swap/add when below)
- **Position concentration:** No single position may exceed 20% of total portfolio value (rule books)
- **Long-only:** unexpected Alpaca shorts → global mismatch halt
- **Fresh holdings at LLM sell:** re-read DB at execute; clamp sells to held shares (June 15 short incident)
- **Active-book Alpaca sync:** pre-trade + post-trade verify; self-heal stale flags (July 22)

8. Risk Management — Trailing Stops

Every held position automatically receives a style-specific trailing stop. The stop only ever moves upward (high-water-mark ratcheting) — never down. `stop_check.py` runs every 15 minutes during market hours with a single yfinance batch call (near-zero cost). Under extreme stress a one-line `tighten_stops(factor)` call can halve all trail percentages instantly. This combination of ratcheting logic + frequent lightweight checks is one of the most important capital-preservation mechanisms in the entire system.

The architecture has three defensive layers, hardened in May 2026 after a bug-class audit found 22 of 60 held positions had broken (stale or triggered) trailing stops:

Layer 1 — Code prevention. Every sell path (autonomous_trader rotation, trailing-stop trigger, trade_executor manual) DELETes the trailing_stops row when the position fully closes. New buys create a fresh active stop via `initialize_stops` which DELETes any leftover row first, then INSERTs (replacing the prior INSERT OR IGNORE that silently dropped new stops when stale rows already existed for the ticker).

Layer 2 — Invariant assertion. `stop_check.py` runs `assert_invariants()` after each maintenance pass. If any held non-SGOV position lacks an active stop, OR any active stop exists for a ticker no longer held, a Slack DM alert fires loudly. Surveillance catches anything Layer 1 misses.

Layer 3 — Self-heal. `remove_stale_stops()` actually DELETes orphan rows (was previously just marking them `status='stale'` and leaving them in place — that buggy pattern blocked new INSERTs and is what produced the original 22 broken stops). The autonomous_trader also calls `initialize_stops` at the end of its run so positions bought at 10:00 CT don't sit unprotected for up to 15 minutes waiting for the next stop_check fire.

The same orphan invariant for `holdings` is checked by `data_health_check.py` every 2 hours and at end of each autonomous_trader run.

8.1 Trail Percentages by Portfolio

Portfolio	Trail %	Rationale
Value Picks	18%	Trust the thesis, wide band
Innovation Fund	18%	Volatile by nature
Income Dividends	15%	Moderate protection
Growth Value	12%	GARP discipline
Nuclear Renaissance	12%	Thematic structural
AI Defense & Autonomous	12%	Thematic structural
Momentum Growth	10%	Cut losers fastest

8.2 Stop Lifecycle

1. **Initialize:** When a new holding is added (or buy completes via `autonomous_trader`), `initialize_stops` DELETes any leftover row for the ticker and INSERTs a fresh active stop with $HWM = \max(\text{current_price}, \text{avg_cost})$. This always produces a clean active row even if a stale entry existed from a prior position.
2. **Ratchet:** Every 2 hours (price refresh) and every 15 minutes (stop check), if $\text{price} > HWM$, the HWM moves up and the trigger price moves up proportionally.
3. **Trigger:** If $\text{price} \leq \text{trigger_price}$, position is sold at market and the `trailing_stops` row is DELETED in the same transaction.
4. **Tighten:** During market stress, `tighten_stops(factor)` cuts all trail bands (e.g., $\text{factor}=0.5$ cuts 18% to 9%). Circuit-breaker mechanism.
5. **Cleanup:** When a position is sold by any path (rotation, trim, manual, trigger), the matching `trailing_stops` row is DELETED in the same DB transaction. `remove_stale_stops()` also runs each cycle and DELETes any orphan rows.

8.3 Intraday Stop Monitor (`stop_check.py`)

Lightweight Python script that runs every 15 minutes during market hours via system crontab. Zero LLM token cost.

- Fetches prices for all held tickers in one `yf.download` batch call (~3 seconds)
 - Runs full stop cycle: `remove_stale_stops` (DELETes orphans), `initialize_stops` (creates new), `ratchet_stops` (move HWMs up), `assert_invariants` (Slack alerts on coverage gap), `check_triggers` (fire stops)
 - If a stop triggers: executes Alpaca market sell, updates DB (DELETes holdings + DELETes `trailing_stops` + INSERTs sell transaction), posts to Slack
 - Enforces same 10 AM – 4 PM ET trading window as all other components
-

Stop-Sell Cooldown (Anti-Round-Trip)

When a trailing stop fires, the IPS scoring engine often still flags the same ticker as top-10 minutes later, leading to immediate rebuy at near-identical price. The May 1 2026 AI Defense incident — LMT/NOC/RTX sold and re-bought within 73 minutes for ~\$4.9K of crystallized losses — exposed this.

The `stop_cooldowns` table records every (portfolio, ticker) that exits via trailing stop, along with the score-at-trigger from the latest `signals.json`. The buy gate in `autonomous_trader._execute_buy_order`'s caller blocks rebuys until **BOTH**:

1. **At least 10 days have elapsed** since the stop trigger.
2. **The current score is at least 2 points above the score at trigger.**

If score never recovers, the position stays blocked indefinitely — the right behavior for a genuinely deteriorated name. The cooldown row is auto-deleted the moment both conditions clear, so the gate is single-firing.

The gate sits at the rotation buy site only — it does not block sells, manual `trade_executor` buys, or rebalance trims. Implementation: `scripts/stop_cooldown.py`.

Order Fill Discipline

`scripts/order_fill.py` is the single source of truth for waiting on Alpaca order fills. The May 6 2026 ANET incident exposed the bug class: a sell order for 73 shares filled fully on Alpaca, but `trailing_stop_manager.py` checked the order one second later, saw 37 of 73 filled (status `partially_filled`), substring-matched on "filled," and recorded only 37 shares to the DB. Alpaca filled the remaining 36 a moment later — DB never updated, \$5,281 of cash drift.

`wait_for_fill()` enforces a strict contract: **the returned** `(filled_qty, filled_price)` **match Alpaca's terminal state for the order, period.** Implementation:

- Polls every 2s up to `timeout_s` (default 30s) for terminal status (`filled` / `canceled` / `expired` / `rejected` / `done_for_day`).
- On primary timeout, requests cancel of the open remainder.
- Polls every 0.5s for up to 10s after cancel (cancel is async; the open quantity can race-fill or race-cancel — both produce a deterministic terminal state).
- Returns the filled quantity at the terminal moment. No "Alpaca fills more shares later that the DB doesn't see" drift is possible.

All five fill-recording paths use this module: `autonomous_trader._execute_sell_order`, `autonomous_trader._execute_buy_order`, the REBALANCE-SELL block, the in-trader STOP-SELL block, `trailing_stop_manager.execute_stop_sells`, and `trade_executor.place_order`. Zero local copies of fill-wait logic remain.

Screening Method — Consistent Application

The screening pipeline has exactly three stages and N-selection happens only at the third:

1. **Finviz query** (`candidate_screener.py`) — broad sector/cap/valuation filter producing raw candidates.
2. **IPS gates** (`style_gates.py`) — hard per-portfolio filters (Buffett value, Wood innovation, O'Neil momentum, etc.). Every ticker that passes a portfolio's gates is a legitimate candidate for that portfolio.
3. **Decision-engine scoring + top-N selection** — every gate-passing ticker is scored on the 20-dimension scoring model. The trader holds the top 10 by score; everything else is a candidate the model can rotate into.

Hard rule: stage 2's output is the universe. There is no cap, no truncation, no secondary filter between stages 2 and 3. Selection by score happens once, in stage 3.

The May 8 2026 alphabetic-bias incident was caused by a hidden stage 2.5: a `MAX_CANDIDATES_PER_PORTFOLIO = 20` cap combined with `sorted()` iteration, which silently kept only the alphabetically-first 20 of however many gate-passers existed. Innovation Fund's 158 gate-passers were reduced to 20 starting A or B; 138 valid C-Z candidates were never seen by the scoring engine. Same bug bit Income Dividends, Value Picks, and Growth Value with varying severity.

The fix removes the cap entirely. Decision-engine compute scales linearly with universe size; current measurement: ~1.4s per ticker, ~400-500 unique tickers across 7 portfolios after the fix → ~10-12 minute decision-engine runs (vs ~4 min before). The cron schedule has 60-150 minute gaps between runs — comfortably accommodates the slower runs.

If gate-pass counts grow beyond what compute permits, the answer is parallelization or batching (Finviz scraping is the bottleneck), not reintroducing a screener cap. **A cap that drops gate-passers by criteria other than the gate criteria is by definition inconsistent screening.**

Accounting Integrity — Three Invariants

The simplified accounting model: **transactions are the single source of truth**. Per-portfolio cash and holdings are recomputable views of the transaction log, not independently mutated state. The audit verifies this with three invariants that must always hold:

Invariant 1 — Per-portfolio cash double-entry

$$\text{starting_cash} - \Sigma(\text{buys.total_value}) + \Sigma(\text{sell.s.total_value}) = \text{current_cash}$$

Tolerance: \$0.01. Failure means the shadow ledger has drifted from its own transaction log (a write went wrong).

Invariant 2 — Active-DB shares match Alpaca shares (per ticker)

$$\Sigma(\text{holdings.shares for is_active}=1 \text{ portfolios}) \text{ for ticker } T \approx \text{Alpaca position qty for } T$$

Tolerance: material share drift on **active** books only. Failure means a buy/sell did not record the right qty (partial-fill bug, lost write, or double-write).

Alpaca orphans (July 22 2026): positions at Alpaca with **no active-DB row** (e.g. leftover CPRX after a sleeve flatten when the asset is broker-inactive and cannot be sold) are **WARN only** — they must not freeze the entire trading system. Documented in `data/alpaca_orphans.json` when cleared from DB.

Invariant 3 — Aggregate cash matches Alpaca account cash

$$\Sigma(\text{per-portfolio current_cash}) \approx \text{Alpaca.account.cash} - \text{unclaimed_setup_cash}$$

Tolerance: \$5 noise. The Alpaca paper account may have pre-existing setup cash that no portfolio claims. INV3 alone is a warning, not always a hard halt (paper setup cash can fail INV3 definitionally).

`scripts/accounting_audit.py` runs all three daily. Critical fail (INV1 or INV2 on **active** books) writes `logs/ALPACA_MISMATCH.flag` and posts CRITICAL Slack.

Pre/post-trade Alpaca guard (July 7 2026; hardened July 22 2026). `verify_account_synced()` in `autonomous_trader.py` runs before order placement on all execute paths (rule trader, Commando, watchers, stop_check, trade_executor). Scope = **active portfolios only**. Halt on: unexpected Alpaca shorts, or many active-DB tickers missing at Alpaca (paper-account wipe signature). **Self-heal:** if the flag exists but halt conditions have cleared, any caller clears the flag (not only the daily autonomous_trader cron). **Post-trade:** `post_trade_verify_or_flag()` re-checks after Commando fills. Path constant: `~/bigclaw-ai/logs/ALPACA_MISMATCH.flag` only (do not invent alternate paths).

Why this matters: in May 2026 the trailing-stop partial-fill bug silently siphoned \$5,281 from Momentum Growth (recording 37 shares sold when Alpaca filled 73). The existing reconciliation only checked share counts (Invariant 2) — both DB and Alpaca read 0 shares post-sell, the share check passed, the cash discrepancy went undetected. Invariant 1 + 3 together would have caught it the next day.

The May 11 2026 reset (`scripts/full_reset_monday.py`) liquidates all positions, wipes the shadow ledger, and re-establishes each portfolio at \$100K starting cash. Both ledgers start equal; the audit verifies they stay equal.

Dashboard same-day-buy bug (May 26 2026 — fixed): the dashboard's per-portfolio `dailyReturn` field was credited with market moves that happened BEFORE we owned a position. For each holding, the dashboard computed `prev_value = shares * prev_close` (yesterday's close). For positions opened on the current day,

`prev_close` is a price from before the position existed, so the daily-return calc captured gains the portfolio never actually realized. Surface symptom: after a 6-position rotation today, Momentum Growth showed +4.32% daily gain while the broader market was flat. MU alone contributed +\$1,620 of phantom gain because it gapped from \$751 to \$875 over the Memorial Day weekend before BigClaw bought it. Fix: when `first_bought_at` equals today's date, use `avg_cost` (the fill price) as `prev_price`. New positions then contribute only their real intraday move since fill. After the fix, MG's daily return resolved to a realistic +0.65%. Prior-day positions are unaffected — their `prev_close` falls within the holding period.

IV tracker — 30-day data-gathering experiment (started May 31 2026): `scripts/iv_tracker.py` runs daily at 11:00 CT via system crontab. Captures forward-looking options-implied volatility signatures (skew = OTM-put-IV minus ATM-call-IV; spread = ATM-call-IV minus ATM-put-IV; classification = BULLISH/MIXED/BEARISH) at ~30d and ~60d expiries for all holdings plus the top-10 scored candidates per portfolio (~80-130 unique tickers/day). Data lands in the `iv_history` SQLite table. Motivation: a literature review of UW/options-flow predictive signals identified IV skew and call-put IV spread as the only signals with documented forward-equity-return edge for our long-only days-to-months horizon — and they're derivable free from yfinance, no UW subscription required. A one-shot diagnostic across current holdings showed +1.10% mean excess vs SPY for BULLISH-IV names vs -1.94% for BEARISH (~3% spread). This experiment gathers enough point-in-time data to validate or refute that spread across ~20 trading days and ~80 tickers, then inform a clean decision-engine integration (or not) after the validation window. Does NOT modify the decision engine. Pure collection.

SHOP partial-trim bug (May 21 2026 — fixed): a SAFETY TRIM 50% on SHOP sold 57 of 114 shares correctly at Alpaca, but the DB deleted the entire holdings row. Root cause: every sell call site computed `sell_all = (filled_qty >= requested_sell_qty)`, which is True for any partial trim where Alpaca fills the full requested amount. For TRIMs the requested qty is half the position, so the row got deleted even though half the shares remained. Fix: `trade_recorder.record_trade` now ignores the `sell_all` parameter and always decrements with cleanup-when-zero. Computing remaining shares from the DB itself is the only reliable source of truth; caller-supplied flags were a footgun. Caught by Invariant 2 (DB shares vs Alpaca shares) the next audit cycle.

Target-Price Capture at Entry

Every new position records `target_price` on the holdings row at entry time. Source: yfinance `info["targetMeanPrice"]` — analyst consensus 12-month target. Also recorded: `target_set_at` (timestamp) and `target_source` ("yfinance_mean").

Semantics: - **Captured on first buy** of a position (when no holdings row exists yet) - **Preserved on subsequent adds** — adding to the position does not overwrite the original target - **Cleared when position fully closes** (holdings row deleted)

The target is captured now (schema migration 2026-05-11) so future Phase 2 work can activate a target-price hold discipline without retrofitting historical data. Phase 2 will define rules like "hold until `current_price >= target_price * 0.90`, sell only on thesis break before then, or trim if position > 15% of portfolio."

Phase 2 is not yet active — daily-rotation discipline remains the rule. But every position bought from 2026-05-11 forward has the target persisted, so when Phase 2 flips on we have the data we need.

Entry-Timing Signals (May 14 2026)

Five signal categories that shift the scoring engine from late-cycle momentum confirmation toward earlier-stage entry detection. Goal: stop the engine from buying stocks at all-time highs where the score is high precisely because the move is already over.

Extension penalties (penalize buying at top-of-rally): - `Extension90d`: penalty when stock is up > 15% in last 90 days (graduated: -0.5 to -2) - `ExtensionMA`: penalty when stock is > 15% above 200-day MA (graduated: -1 to -2)

Breakout rewards (reward buying at start-of-rally): - `Breakout`: reward when price freshly punches above 60-day high (+1 to +2) - `VolContract`: reward when 30-day volatility is in bottom quartile vs 1-year (+1 to +2) - `MAReclaim`: reward when stock freshly reclaims 50-day MA after being below (+2)

Style-tuned weights (in `STYLE_WEIGHTS`): - Innovation Fund / Value Picks / Growth Value / Income Dividends: extension 1.5×, breakout 1.5× - Momentum Growth: extension 0.5× (riding momentum is the strategy), breakout 2.0× (catch early) - Nuclear / AI Defense: 1.0× across all

Validation status: backtested against 5 pre-reset winners and 6 recent losers. Breakout signals correctly boosted DDOG/ARGX/AMBA at their entry points. Extension penalty correctly flagged ANAB at +57% as extended (though ANAB continued running — false positive on this case). The recent losers (HUBS/PODD/RBLX) were already falling at entry — different bug class (falling-knife detection) not addressed by these signals. That work belongs to a future phase.

Research Dossier Pipeline (Phase A)

`scripts/research_dossier.py` generates one-page Markdown research dossiers for candidate tickers. Pulls data from all available sources (yfinance info, eps_trend, analyst revisions, signals.json, holdings, target_price) and uses Claude Sonnet 4.6 to synthesize into a structured doc.

Output format: - Signal snapshot (BigClaw score, analyst consensus, EPS revisions, valuation) - Bull case (3-4 specific points with data citations) - Bear case (3-4 specific points) - Key unknowns (questions a domain expert should ask) - One-line recommendation

Output location: `docs/dossiers/{ticker}_{YYYYMMDD}.md`

Cost: ~\$0.02-0.03 per dossier (Sonnet input + output). 70 dossiers/week = \$1-2/week or ~\$78/year — comfortable within R&D budget.

Usage: - Single ticker: `python3 scripts/research_dossier.py --ticker AMD` - Whole portfolio: `python3 scripts/research_dossier.py --portfolio "Innovation Fund"` - All top-10: `python3 scripts/research_dossier.py --top10` - Data dump only: `python3 scripts/research_dossier.py --ticker AMD --dry-run`

Phase A (manual invocation only) ships May 15. Phase B (Slack review UI) and Phase C (trader gates on Curtis approval) deferred until Phase A is used in practice for 2-4 weeks to validate the format and information density.

9. Scheduled Operations & Automation

All recurring work is defined in OpenClaw's `cron/jobs.json`. Disabled jobs are explicitly listed so no one accidentally re-enables them.

9.1 OpenClaw Cron Jobs (Weekday)

All times Eastern unless noted.

Time	Job	Model	Purpose
8:55 AM	Morning Data Gather	Gemini Flash Lite	Runs <code>morning_data_gather.py</code> , writes raw data to <code>/tmp/bigclaw_morning_data.txt</code>
9:00 AM	Morning Market Analysis	Claude Sonnet	Reads morning data file, produces market analysis + portfolio implications, posts to Slack
~~9:00, 11:00, 1:00, 3:00 PM~~	~~Price Refresh (2hr)~~	—	DISABLED May 28 2026. Was an agent-turn cron that ran <code>price_refresh.py</code> via the LLM exec tool; broke when the OpenClaw exec-approval socket stopped being connected (every run pinged Curtis for approval). Redundant with <code>refresh_all.sh</code> (system cron, runs <code>price_refresh.py</code> 5x/day directly). Price refresh now runs solely via the native bash cron.
10:30 AM	Daily Autonomous Trading	Gemini Flash Lite	Runs <code>autonomous_trader.py</code> — full decision engine + trade execution cycle
4:25 PM	Afternoon Data Gather	Gemini Flash Lite	Runs <code>afternoon_data_gather.py</code> , writes raw data to <code>/tmp/bigclaw_afternoon_data.txt</code>
4:30 PM	Afternoon Portfolio Report	Gemini Flash Lite	Reads afternoon data file, produces portfolio performance report, posts to Slack

9.2 OpenClaw Cron Jobs (Weekly)

Schedule	Job	Model	Purpose
Saturday 7:45 AM CT	ARK ITK Summary	Python (zero LLM)	<code>ark_itk_tracker.py</code> (system cron) generates the summary; <code>post_ark_itk.py</code> posts it to Slack natively. Migrated off the OpenClaw/Gemini-Flash-Lite agent-turn (which kept failing 'Message failed' — the agent only ever cat'd the file verbatim, so no LLM was needed) June 29 2026
Saturday 8:00 AM CT	Weekly Research Session	Claude Sonnet	Autonomous deep research into market themes and opportunities
Saturday 9:00 AM CT	Candidate Screener	Python (zero LLM) + Opus 4.6 (borderline)	Discovers new candidate stocks via Finviz, filters through style gates + AI reasoning, updates <code>portfolio_universes.json</code>
Saturday 9:00 AM CT	Weekly Style Compliance Audit	Gemini Flash Lite + Opus 4.6 (borderline)	Audits holdings against style gates with AI reasoning for borderline cases
Sunday 7:00 AM CT	Weekly Network Security Scan	Gemini Flash Lite	Runs security scan of the Pi
Sunday 9:00 AM CT	Weekly OpenClaw Version Check	Gemini Flash Lite	Checks for OpenClaw updates and model version changes

9.3 OpenClaw Cron Jobs (Daily)

Time	Job	Model	Purpose
7:00 AM CT daily	Good Morning + Security Check	Gemini Flash Lite	System health check, security scan

9.4 System Crontab Jobs (selected; see live `crontab -l` on Pi)

Schedule (CT)	Script	Purpose
0 9, 11:30, 14:30 weekdays	llm_comando.py --cycle ...	Commando full dialectic sessions
* /5 8-15 weekdays	llm_comando_watcher.py	Intraday triggers + mechanical exits
* /2 8-15 weekdays	llm_comando_radar.py	Event radar + price_is_news
50 14 weekdays	llm_comando_reconciler.py	End-of-day exit backstop
every 15 min RTH	stop_check.py	Trailing stops (all books)
various	rule DE + autonomous_trader	Active rule sleeves

9.4b System Crontab Jobs (legacy heading retained)

These jobs run via the Pi's system crontab — pure Python, zero LLM cost.

Schedule	Script	Purpose
7:30 AM CT, weekdays	daily_export.sh	Pre-market full export: sector heatmap, calendar, trades (last 2 days), analysis, news, performance chart, all per-ticker charts. Alerts Slack on failure.
9:00, 10:00, 12:00, 2:00, 4:30 PM CT, weekdays	refresh_all.sh	Signals + prices + charts refresh: runs decision engine with rescreen (<code>export_signals.py</code>), macro scanner, price refresh (<code>price_refresh.py</code>), and per-ticker chart export (<code>export_charts.py</code>). Enriches each signal with portfolio holding context and swap recommendations. Commits and pushes to GitHub Pages. Alerts Slack on failure.
9:30 AM CT (10:30 AM ET), weekdays	autonomous_trader.py	Autonomous trade execution: syncs DB with Alpaca, runs decision engine with rescreen, executes sells/swaps/buys, post-trade sync check. The 9:00 AM signal refresh provides a 30-minute supervisory window before trades execute.
Every 15 min, 9 AM–3 PM CT, weekdays	stop_check.py	Intraday trailing stop monitor (zero LLM cost)

9.5 Disabled Jobs

Job	Reason
Tuesday Trading Execution	Replaced by Daily Autonomous Trading
Good Morning Curtis	Replaced by Good Morning + Security Check
Email Check: fixit@grandpapa.net	Paused

10. Data Integrity & Anti-Hallucination Pipeline

LLMs are never allowed to fabricate numbers. Every market report follows a rigid two-step process:

Step 1 — Data Gather (Python, No LLM)

Pure-Python data-gather scripts (`morning_data_gather.py` , `afternoon_data_gather.py`) pull every price, fundamental, and statistic and write them to flat text files with clear `=== SECTION ===` delimiters. Every number, price, and percentage is captured from live APIs before any LLM is involved.

Step 2 — Analysis (LLM Reads File Only)

The LLM is instructed to read only that file. The prompt enforces critical rules:

- Every number must appear verbatim in the data file
- No web search, no guessing, no fabrication
- If data is missing, state "data unavailable"

Morning Market Brief refresh (June 18 2026). The Slack morning brief had been rendering dead Unusual Whales sections (GEX / options flow, market tide, smart-money / congressional, insider) as "stale / unavailable" ever since the UW cancellation on May 31 — the data gather no longer produced them, but the prompt still asked for them. The brief was rebuilt around the feeds BigClaw actually uses: a **DECISION ENGINE** section (`signals_brief.py`, from `signals.json` — planned buys/sells, top-scored candidates, executed-this-week), an **IV SIGNAL** section (`iv_brief.py`, the 30-day forward options read from `iv_history` — overall BULLISH/MIXED/BEARISH tally plus per-holding classification and the strongest bullish/bearish names, the one options signal with documented forward-return edge that was kept when UW was cut), and the macro-**REGIME** tells (VIX, HY-vs-IG credit — `macro_prices.py` now also pulls $\wedge$ VIX / HYG / LQD). Polymarket is restricted to macro / geopolitical markets (skip consumer / meme noise).

Runtime Guardrails

Two runtime guardrails reinforce this:

- **Price Oracle** (`src/services/price_oracle.py`): Single source of truth for all prices system-wide. In-memory cache with 10-minute TTL. Refuses to serve stale data.
- **Output Guardrail** (`src/services/output_guardrail.py`): Scans every outbound message for `$xxx.xx` patterns and corrects any deviation > 2% against the oracle.

This pipeline is the reason BigClaw's reports remain trustworthy even when LLMs are involved.

Trade Recording Integrity

All sell paths (`autonomous_trader` rotation, trailing-stop trigger, `trade_executor` manual) wait for Alpaca fill confirmation, then record the actual `filled_avg_price` into the `transactions` table — not the originally submitted limit price. This was hardened May 2026 after a cash-drift audit found ~\$47K of accumulated misrecording from older trades that had been logged at limit price; limit orders fill at-or-below limit, so historical trades systematically over-recorded cash spent.

Historical drift is bounded (no new contributions). Per-portfolio cash accounting (`starting_cash - SUM(buys) + SUM(sells) + SUM(dividends)`) is correct going forward — each portfolio sees only its own trades, and the cash wall isolation ensures one portfolio cannot spend another's cash.

LLM Token Surveillance

`scripts/llm_token_report.py` runs daily at 6 AM CT, aggregates the previous day's calls from `logs/llm_calls.jsonl`, and posts a Slack DM with per-script breakdown. Alerts loudly if any single script exceeds 1M tokens or \$5 in one day. Catches the pattern of a heavy LLM consumer creeping into the codebase undetected — the way `gate_reasoning` did before it was found burning ~\$1,460/year.

Symbol Format at the Alpaca Boundary

BigClaw stores tickers using the Yahoo/Finviz convention with hyphens for class-share suffixes (BRK-B, BF-A, BF-B, MOG-A). Alpaca uses dots (BRK.B, BF.A). `scripts/alpaca_symbols.py` provides `to_alpaca()` and `from_alpaca()` for translation at the API boundary — applied at every `submit_order` site (autonomous_trader, trade_executor, trailing_stop_manager, bigclaw_full_reset) and at `get_all_positions()` reads (so reconciliation compares apples to apples). The DB and the rest of the codebase only ever see the hyphen format. Added May 4 2026 after Income Dividends BUY orders for BF-A and BF-B were rejected by Alpaca with `asset not found`.

11. Data Feeds & External APIs

11.1 Market Data & Pricing

Source	Library/API	Data Provided
Yahoo Finance	<code>yfinance</code> 1.1.0	Price history, fundamentals, earnings dates, dividends, options chain, per-ticker news
Finviz	<code>finvizfinance</code> 1.3.0	Short interest, insider transactions, analyst ratings, price targets
Alpaca	<code>alpaca-py</code> 0.43.2	Paper trade execution, account state, position sync, real-time/extended-hours quotes
FRED	CSV API (free)	Michigan Consumer Sentiment, Personal Saving Rate, Treasury yield curve, credit spreads

11.2 Options Flow & Institutional Intelligence

Source	Method	Data Provided

Unusual Whales subscription cancelled May 31 2026. A deep-research pass over the academic literature found that UW's flagship signals (flow, dark pool, GEX, market tide, congressional trades) have no documented forward-equity-return edge for our long-only days-to-months horizon, and a point-in-time backtest on our own buys agreed (no positive spread between bullish-UW and bearish-UW picks). The one signal with documented edge (IV skew + call-put IV spread) is derivable for free from `yfinance` option chains and is now collected daily by `iv_tracker.py`. The decision engine never used UW data, so trade execution is unaffected. Scripts removed: `options_intelligence.py`, `unusual_whales.py`, `uw_api_extended.py`, `tsla_watchdog.py`. Data gathers `morning_data_gather.py` and `afternoon_data_gather.py` had their UW sections removed; the rest of their data continues to feed the Slack briefings.

11.3 Sentiment & Social

Source	Method	Data Provided
X/Twitter	Apify scraper API	Real-time ticker sentiment from financial Twitter
StockTwits	Apify scraper (fallback)	Retail sentiment per ticker
Reddit (WallStreetBets)	Direct API	Trending tickers, retail sentiment
CNN Fear & Greed Index	JSON endpoint (free)	Composite market sentiment score (0-100), used as contrarian indicator

11.4 News & Research

Source	Method	Data Provided
Motley Fool	RSS feed (feedparser)	Investing and market news via four RSS feeds
Yahoo Finance News	yfinance	Per-ticker headlines pulled alongside price data
Brave Search	REST API (API key)	On-demand web search for fact-checking, catalyst verification, and research queries
SEC EDGAR	REST API (edgartools)	Insider trades (Form 4), annual reports (10-K), regulatory filings
Polymarket	Gamma API (free)	Prediction market odds on economic/political events

11.5 ARK Invest Tracking

Source	Method	Data Provided
CathieArk.com	Web scrape	Daily ARK fund trade notifications — buys, sells, position changes across ARKK, ARKG, ARKW
ARKFunds.io	REST API (free)	Structured ETF trade data from ARK's daily disclosures (primary source)
YouTube / ARK ITK	YouTube Data API + youtube-transcript-api	Weekly "In the Know" video — auto-discovered, transcribed, and summarized

ARK tracking feeds the Innovation Fund portfolio directly and is summarized weekly via the Saturday ARK ITK cron job.

11.6 Economic Calendar & Macro

Source	Method	Data Provided
Federal Reserve	FOMC calendar scrape	Interest rate decision dates and policy meeting schedule
Bureau of Labor Statistics	Schedule scrape	CPI inflation and Nonfarm Payrolls release dates
Bureau of Economic Analysis	Schedule scrape	GDP release schedule — quarterly growth data
Yahoo Finance (Earnings)	yfinance	Next earnings dates and EPS estimates for portfolio holdings

11.7 Weather & Environment

Source	Method	Data Provided
Open-Meteo	REST API (free, no auth)	Daily weather forecasts for Alvarado, TX — included in morning briefings

11.8 Website Frontend

Technology	Purpose
Chart.js	Portfolio performance line charts (CDN via jsDelivr)
TradingView	Embedded symbol overview widget for live ticker lookup
Google Fonts (Inter)	Typography across all dashboard pages
GitHub Pages	Static site hosting — files pushed from Pi via git, zero infrastructure cost

11.9 LLM Providers

Provider	Models Used	Purpose	Cost Model
Anthropic (direct)	Claude Opus 4.6	Interactive development via Claude Code; style gate AI reasoning for borderline decisions	Pro subscription + API (~\$20–30/yr for gates)
Anthropic (direct)	Claude Sonnet	Interactive Slack conversations	Pro subscription
OpenRouter	Claude Sonnet 4.6	Morning analysis, weekly research (high reasoning)	~\$0.20/call
OpenRouter	Gemini 3.1 Flash Lite	Data gather, price refresh, trading (low cost)	~\$0.01/call
OpenRouter	Gemini 2.5/2.0 Flash Lite	Fallback models	~\$0.005/call

Fallback chain: Gemini 3.1 Flash Lite → Gemini 2.5 Flash Lite → Gemini 2.0 Flash Lite

11.10 Communication

Service	Purpose
Slack (Bot Token + App Token)	Primary interface — receives reports, trade notifications, interactive queries
Discord (Webhook)	Error alerts, synchronized reports

11.11 Technical Analysis Libraries

Library	Version	Purpose
<code>ta</code>	0.11.0	RSI, MACD, SMA, Bollinger Bands, technical indicators
<code>pandas-ta</code>	0.4.71b	Extended technical analysis
<code>pandas</code>	3.0.0	Data processing, time series
<code>numpy</code>	2.2.6	Numerical computation
<code>matplotlib</code>	3.10.8	Chart generation (performance, MACD, RSI, Bollinger, Monte Carlo)
<code>ffn</code>	1.1.2	Financial analysis (Sharpe ratio, max drawdown, risk metrics)

The complete, browsable list of all data sources with links and descriptions is maintained on the dashboard's Sources page at bigclaw.grandpapa.net/sources.html.

12. User Interfaces

12.1 Slack — Primary Interactive Channel

The Slack bot runs as `bigclaw.service` and exposes 36 interactive tools that the Claude agent can invoke during conversation. Users interact via direct message or channel mention; the agent maintains per-channel conversation memory. The full tool inventory is documented in Section 3.2.1.

12.2 GitHub Pages Dashboard

Hosted on GitHub Pages at **bigclaw.grandpapa.net** behind Cloudflare Access. Data is refreshed on two schedules:

- **5× daily** (9 AM, 10 AM, 12 PM, 2 PM, 4:30 PM CT) via `refresh_all.sh`: signals, prices, macro, charts
- **Intraday price-only** (~every 30 min, 10:15–14:30 CT) via standalone `price_refresh.py`: prices + portfolio values only, LLM-free, for near-live dashboard freshness between the `refresh_all` anchors
- **1× daily pre-market** (7:30 AM CT) via `daily_export.sh`: sector heatmap, calendar, trades, analysis, news, performance chart

Both scripts alert Slack on failure so stale data is never silent.

Page	File	Purpose
Home	<code>index.html</code>	Market overview, portfolio summary cards
Dashboard	<code>dashboard.html</code>	Today's Planned Actions (sells and strong buys only), portfolio cards, sector rotation heatmap, upcoming events, recent trades, market analysis
Portfolio	<code>portfolio.html</code>	Individual portfolio deep dive — holdings table, metrics, P&L
Signals	<code>signals.html</code>	Supervisory display and single source of truth for all signal categories: action required, watch/caution, opportunities, hold. Each signal shows portfolio holding tags (shares, position %, capacity status) and pending swap recommendations with score differentials. Also: macro overview, sentiment, bond market, alerts & warnings
Ticker	<code>ticker.html</code>	Individual ticker detail with candlestick, MACD, RSI, Monte Carlo charts + signal context
Chart Detail	<code>chart-detail.html</code>	Portfolio performance comparison chart
Sources	<code>sources.html</code>	Data source documentation, cron schedule reference
Privacy	<code>privacy.html</code>	Privacy policy
Nav	<code>nav.html</code>	Shared navigation component
Styles	<code>styles.html</code>	Shared CSS

Design rule: The dashboard shows only actionable items (planned trades). All detailed signal analysis lives on the signals page. This avoids redundancy and ensures the signals page is authoritative. The signals page serves as the supervisory display — like Tesla FSD (Supervised), the autonomous trader makes all decisions but the human sees the same data and reasoning, with a 30-minute window before trade execution to intervene if needed.

12.3 Dashboard Data Files

All JSON data in `docs/data/` is auto-refreshed and pushed to GitHub Pages:

File	Contents	Refresh
<code>signals.json</code>	Decision engine output with rescreen: scores, labels, reasons, portfolio holding context (shares, position %, capacity), swap recommendations with score differentials	5×/day
<code>portfolios.json</code>	All 8 portfolios with holdings, current prices, returns	5×/day
<code>llm_portfolio.json</code>	LLM Discretionary daily bull/bear/judge output + trades	1×/day weekday
<code>llm_outcomes.jsonl</code>	Append-only trade closure log: entry/exit, realized %, trigger, prediction accuracy	continuous
<code>llm_pending_triggers.json</code>	Today's armed intraday triggers + fires_today counter	per-cycle, per-fire
<code>llm_comando_portfolio.json</code>	LLM-Commando daily bull/bear/judge output + trades	1×/day weekday
<code>llm_comando_journal.jsonl</code>	LLM-Commando append-only journal	per-cycle
<code>llm_comando_outcomes.jsonl</code>	LLM-Commando trade closure log	continuous
<code>llm_comando_pending_triggers.json</code>	LLM-Commando intraday triggers + fires counter	per-cycle, per-fire

File	Contents	Refresh
sector_rotation.json	Weekly sector rotation report (LLM-synthesized)	1×/week Sunday
market.json	S&P 500, Dow, NASDAQ, VIX + sector rotation heatmap (11 GICS sectors)	5×/day (indices), 1×/day (sectors)
macro.json	Macro indicators, bond yields, sentiment, sector performance, VIX, verdict	5×/day
charts/*.json	Per-ticker OHLCV, MACD, RSI, Monte Carlo for ~50 held tickers	5×/day
trades.json	Recent trade history (last 2 trading days)	1×/day
calendar.json	Upcoming economic events (FOMC, CPI, NFP)	1×/day
analysis.json	Macro market scanner report (markdown)	1×/day
news.json	Financial news headlines (Motley Fool RSS)	1×/day
metadata.json	Last update timestamps, export status	Every refresh
performance_chart.png	Portfolio performance vs S&P 500 (SPY), rebased to Apr 16 2026 refactor baseline	1×/day

Note: `price_refresh.py` preserves sector data in `market.json` when refreshing index prices — it merges rather than overwrites.

13. Logging, Monitoring & Security

Everything is logged. ERROR/CRITICAL entries are mirrored to Discord. API calls use automatic retry logic. Secrets live only in `~/.env_secrets` (never in git). SSH is key-only, `fail2ban` is active, unattended upgrades run, and a weekly security scan is part of the Sunday routine. The GitHub repo is intentionally a read-only review copy — no secrets ever leave the Pi.

13.1 Application Logs

All scripts log via `bigclaw_logging.py`:

- **Log file:** `~/bigclaw-ai/logs/bigclaw.log` (rotated at 5 MB, 2 backups)
- **Format:** `YYYY-MM-DD HH:MM:SS [bigclaw.module_name] LEVEL: message`
- **Discord alerts:** ERROR and CRITICAL level messages are auto-posted to Discord webhook

13.2 Trade Logs

- **File:** `~/openclaw/workspace/logs/trades.log`
- **Format:** `YYYY-MM-DD HH:MM ET | ACTION | portfolio | ticker | details`
- **Actions logged:** BUY, SELL, STOP-SELL, REBALANCE-SELL, SWAP, TRIM, SKIP, ERROR

13.3 Intraday Stop Check Logs

- **File:** `~/bigclaw-ai/logs/stop_check.log`
- **Frequency:** Every 15 minutes during market hours

13.4 API Retry

All external API calls use `bigclaw_retry.py` for automatic retries on transient failures (configurable attempts, delay, and label for logging).

13.5 Configuration Files

File	Purpose
<code>~/.env_secrets</code>	All API keys: Alpaca, Anthropic, Slack, Discord, X/Twitter (Apify), OpenRouter
<code>~/.openclaw/openclaw.json</code>	Agent runtime config: models, channels, gateway settings
<code>~/.openclaw/cron/jobs.json</code>	All 14 scheduled job definitions with schedules, models, prompts
<code>~/.openclaw/workspace/config/portfolio_universes.json</code>	Per-portfolio allowed ticker lists (holdings + candidates), refreshed weekly by candidate screener
<code>~/.openclaw/workspace/config/expert_overrides.json</code>	Manual conviction score overrides for specific tickers
<code>~/.openclaw/workspace/SOUL.md</code>	Agent personality, analytical mandate, behavioral rules
<code>/etc/systemd/system/bigclaw.service</code>	Systemd service definition for the Slack bot
<code>~/bigclaw-ai/.env</code>	Application environment variables (Slack tokens, API keys)
<code>~/bigclaw-ai/.gitignore</code>	Ensures secrets, databases, and caches are never committed

13.6 Security

- All API keys stored in `~/.env_secrets` (not in git, not in any tracked file)
- `.env` and `*.db` files are in `.gitignore`
- SSH access via ed25519 key authentication only
- Weekly automated network security scan (Sunday 7 AM)
- Daily security check as part of morning routine
- `fail2ban` running for brute-force protection
- Unattended upgrades enabled for security patches
- GitHub codebase repo is a read-only review copy — no secrets ever pushed

14. Key Design Principles (Guidance for Anyone Who Comes After)

These nine principles are the operating manual. Violate them at the risk of breaking the system's integrity:

1. **The Pi is the source of truth.** All execution, state, and decisions happen on the Raspberry Pi. Local copies and GitHub are snapshots only.
2. **Zero-token where possible.** Data gathering, price refreshes, trailing stop checks, and trade execution are pure Python — no LLM tokens consumed. LLMs are reserved for synthesis, analysis, and interactive conversation.
3. **Style fidelity above all.** Each portfolio's signal weights are sacred. A stock that's a BUY for Momentum may be a HOLD for Value. Do not apply a generic scoring model across styles.
4. **Consistent trading window.** No trades before 10:00 AM ET or after 4:00 PM ET, weekdays only. Every component must enforce this — at cron level, script level, and Alpaca execution level.

5. **Anti-hallucination by design.** Data files first, LLM second. The 2-step pipeline (Python data gather → LLM analysis of flat files) prevents fabricated numbers. The Price Oracle and Output Guardrail are non-optional.
6. **Sell before buy.** The autonomous trader always executes sells first to free cash, then buys. This prevents over-allocation and ensures cash is available for the best opportunities.
7. **Stops only move up.** Trailing stop high water marks ratchet upward on price increases but never decrease. This is the primary capital-protection mechanism. This locks in gains progressively.
8. **Log everything.** Every trade, every API call, every decision is logged with timestamps and rationale. Post-mortem analysis depends on it.
9. **WAL mode everywhere.** Every SQLite connection uses WAL journal mode and a 30-second busy timeout. Multiple cron jobs overlap during market hours (stop checks, price refreshes, trading). WAL prevents database-locked errors that caused 14 failed trade recordings on 2026-03-30.

This document, the `ARCHITECTURE.md` in the repo, the `SOUL.md`, and the live Pi itself are the complete specification. If you are reading this because you are taking over maintenance, start by running the weekly style-compliance audit and the security scan, then read the logs. The system is designed to be understandable, auditable, and safe — as long as these principles are respected.

Document generated March 30, 2026. Based on live system state from Raspberry Pi "BigClaw" at 192.168.1.171.